

项目实战：AI搜索类应用



>> 今天的学习目标

项目实战：AI搜索类应用

- AI搜索类应用

Version1：对于多文件快速进行检索和回答

Version2：海量文件快速索引（ES）

Version3：添加向量检索功能

Version4：添加外部数据源（AI搜索MCP）

Version5：界面美化

- 基于Qwen-Agent的最佳实践

CASE：长对话检索与问答

CASE：多文档并行问答

CASE：多智能体问答

CASE：多智能体协作路由

CASE： AI搜索类应用

Thinking： 企业对AI搜索问答的需求是怎样的？

- 给客服提供AI辅助， 客服团队往往有200+人， 需要响应及时， 首个token回复一般在 3-5秒内
- 数据源分散， 文档多， 通常会有100+， 甚至更多的文档。大部分文档是关于产品介绍， 活动规则， 常见问题等
- 针对互联网企业， 文档数量会非常巨大， 比如上亿的文档， 并且需要及时检索

Thinking： 企业中RAG上线的要求一般是怎样的？

速度快： 3-5秒内开始出结果

准确率高： 客服辅助准确率要求99.9%以上， 可以不回答， 但是不能出错

Version1: 对于多文件快速进行检索和回答

CASE: AI搜索类应用

TO DO: 使用Qwen-Agent搭建多文件搜索问答应用

Step1, 跑通之前的代码 qwen-agent-multi-files.py

```
59 tools = ['my_image_gen', 'code_interpreter'] # `code_interpreter`
60 import os
61 # 获取文件夹下所有文件
62 file_dir = os.path.join('./', 'docs')
63 files = []
64 if os.path.exists(file_dir):
65     # 遍历目录下的所有文件
66     for file in os.listdir(file_dir):
67         file_path = os.path.join(file_dir, file)
68         if os.path.isfile(file_path): # 确保是文件而不是目录
69             files.append(file_path)
70 print('files=', files)
71
72 bot = Assistant(llm=llm_cfg,
73                 system_message=system_instruction,
74                 function_list=tools,
75                 files=files)
76
77 # 步骤 4: 作为聊天机器人运行智能体。
78 messages = [] # 这里储存聊天历史。
79 query = "介绍下雇主责任险"
```

雇主责任险是一种为企业设计的保险产品，旨在转移企业在雇佣员工过程中可能面临的法律责任和经济风险。险的主要内容和特点：

1. **保障范围**

雇主责任险主要保障以下方面：

- **死亡赔偿金**：员工因工作原因身故，企业需支付的赔偿。
- **伤残赔偿金**：员工因工作原因伤残，企业需支付的赔偿。
- **医疗费用**：员工因工作原因受伤或患职业病产生的医疗费用。
- **误工费**：员工因伤暂时无法工作，企业需支付的工资补偿。
- **法律诉讼费**：因工伤纠纷产生的法律费用。

2. **适用对象**

- 任何有用工需求的单位，包括企业、个体工商户等。
- 覆盖正式员工、临时工、学徒工等（无需正式劳动合同，事实劳动关系即可）。

3. **产品亮点**

- **风险转移**：将企业对员工的责任风险转嫁给保险公司。
- **税前列支**：保费可计入企业成本，减少25%的企业所得税。
- **减少纠纷**：快速理赔，降低工伤纠纷。
- **灵活定制**：可根据企业需求选择不同版本（如A款、B款、94款）。

4. **与其他保险的区别**

- **vs 团体意外险**：
 - 雇主责任险从法律角度保障企业，意外险仅是员工福利。
 - 雇主责任险涵盖误工费 and 职业病，意外险不涵盖。
- **vs 工伤保险**：
 - 工伤保险无法覆盖全部费用（如一次性就业补助金）。
 - 雇主责任险理赔更快捷，不影响企业安全记录。

5. **理赔案例**

- **案例1**：员工上班途中车祸身故，获赔30万元。

CASE: AI搜索类应用

Step2, 在 qwen-agent-multi-files.py 的基础上, 添加GUI

@qwen-agent-multi-files.py 参考 @assistant_ticket_bot-3.py 添加 gui模式, 编写新的 .py

```
qwen-agent-multi-files-gui.py +177 -1 ✓
import pprint
import urllib.parse
import json5
from qwen_agent.agents import Assistant
from qwen_agent.tools.base import BaseTool, register_tool
import os
from qwen_agent.gui import WebUI

# 步骤 1 (可选): 添加一个名为 `my_image_gen` 的自定义工具。
@register_tool('my_image_gen')
class MyImageGen(BaseTool):
    # `description` 用于告诉智能体该工具的功能。
    description = 'AI 绘画 (图像生成) 服务, 输入文本描述, 返回基于文本信息绘
```



```
149 def app_gui():
150     """图形界面模式, 提供 Web 图形界面"""
151     try:
152         print("正在启动 Web 界面...")
153         # 初始化助手
154         bot = init_agent_service()
155         # 配置聊天界面, 列举3个典型门票查询问题
156         chatbot_config = {
157             'prompt.suggestions': [
158                 '画一只在写代码的猫',
159                 '介绍下雇主责任险',
160                 '帮我画一个宇宙飞船, 然后把它变成黑白的'
161             ]
162         }
163         print("Web 界面准备就绪, 正在启动服务...")
164         # 启动 Web 界面
165         WebUI(
166             bot,
167             chatbot_config=chatbot_config
168         ).run()
169     except Exception as e:
170         print(f"启动 Web 界面失败: {str(e)}")
171         print("请检查网络连接和 API Key 配置")
```

CASE：AI搜索类应用

介绍下雇主责任险

介绍下雇主责任险



雇主责任险是一种为企业设计的保险产品，旨在为企业提供保障，以应对员工在工作期间发生意外事故或罹患职业病时可能产生的经济赔偿责任。这种保险可以帮助企业将潜在的雇员风险转嫁给保险公司，从而降低企业的用工风险和减少工伤纠纷。

以下是雇主责任险的一些关键特点：

1. 保障范围:

- 死亡赔偿金
- 伤残赔偿金
- 医疗费用
- 误工费用（部分计划可能不包括）
- 法律诉讼费（部分计划可能不包括）

2. 其他亮点:

- 雇主可以将对员工的风险责任转嫁给保险公司。
- 有助于降低企业用工风险。
- 减少工伤纠纷。
- 提高员工福利保障。
- 某些情况下，保费可以在税前列支，从而减轻企业税务负担。

3. 不同版本的区别:

- A款: 根据《工伤保险条例》的规定，涵盖因工作而遭受的意外伤害或职业病导致的伤、残或死亡。
- B款: 从《工伤保险条例》中摘录的部分工伤情形，涵盖因此类情形导致的伤、残或死亡。
- 94款: 覆盖因工作而遭受的意外伤害或职业病导致的伤、残或死亡，根据雇员工资总额进行



None

I'm a helpful assistant.

插件

☒ my_image_gen

☒ code_interpreter

推荐对话

画一只在写代码的猫

介绍下雇主责任险

帮我画一个宇宙飞船，然后把它变成黑白的

RAG的三级架构 (Qwen-Agent)

搭建RAG (Qwen-Agent)

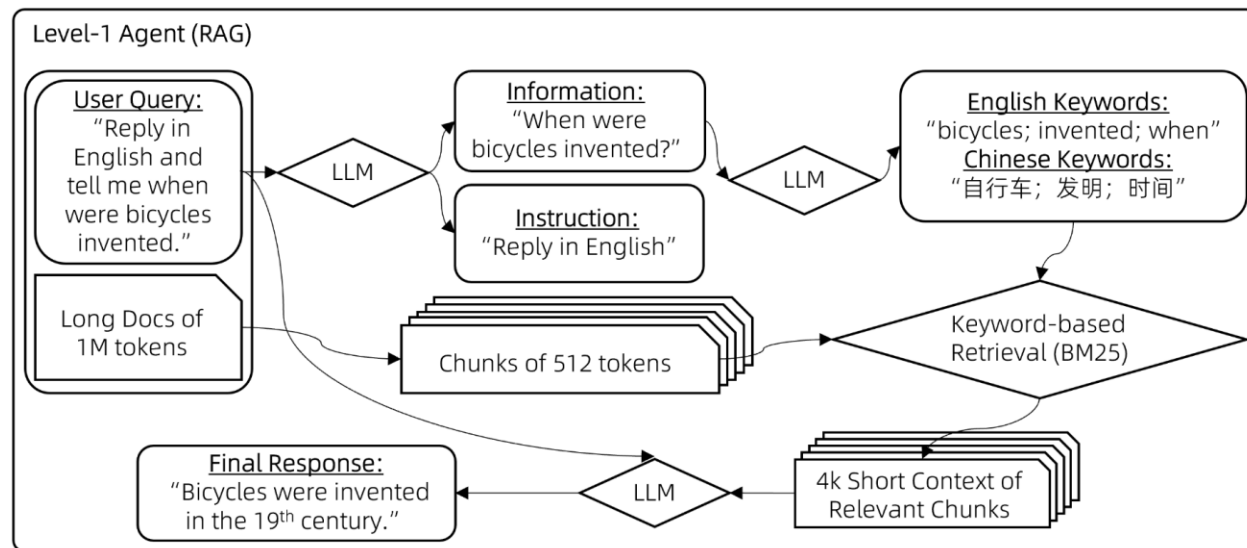
Qwen-Agent构建的智能体包含三个复杂度级别，每一层都建立在前一层的基础上：

- 级别一：检索

处理100万字上下文的一种朴素方法是采用RAG。

RAG将上下文分割成较短的块，每块不超过512个字，然后仅保留最相关的块在8k字的上下文中。

挑战在于如何精准定位最相关的块。经过多次尝试，我们提出了一种基于关键词的解决方案：



级别一：检索（流程图）

搭建RAG (Qwen-Agent)

步骤1：指导聊天模型将用户查询中的指令与非指令信息分开。

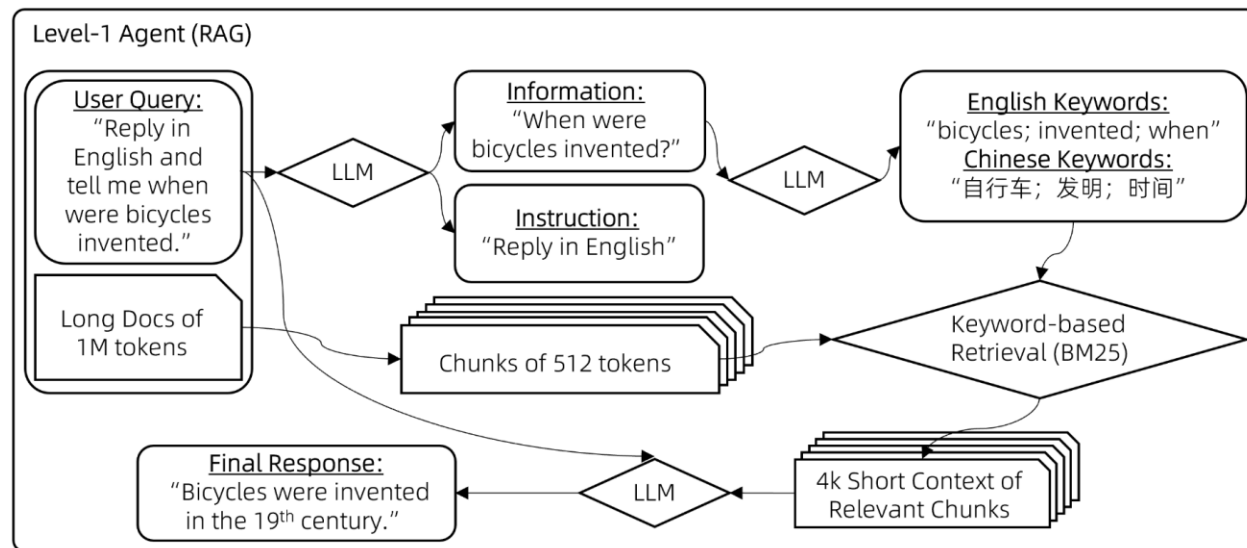
例如，将用户查询"回答时请用2000字详尽阐述，我的问题是，自行车是什么时候发明的？请用英文回复。"

转化为{"**信息**":["自行车是什么时候发明的"], "**指令**":["回答时用2000字", "尽量详尽", "用英文回复"]}

步骤2：要求聊天模型从查询的信息部分推导出多语言关键词。

例如，短语"自行车是什么时候发明的"会转换为{"**关键词_英文**":["bicycles", "invented", "when"], "**关键词_中文**":["自行车", "发明", "时间"]}

步骤3：运用BM25这一传统的基于关键词的检索方法，找出与提取关键词最相关的块。



级别一：检索（流程图）

搭建RAG (Qwen-Agent)

级别二：分块阅读

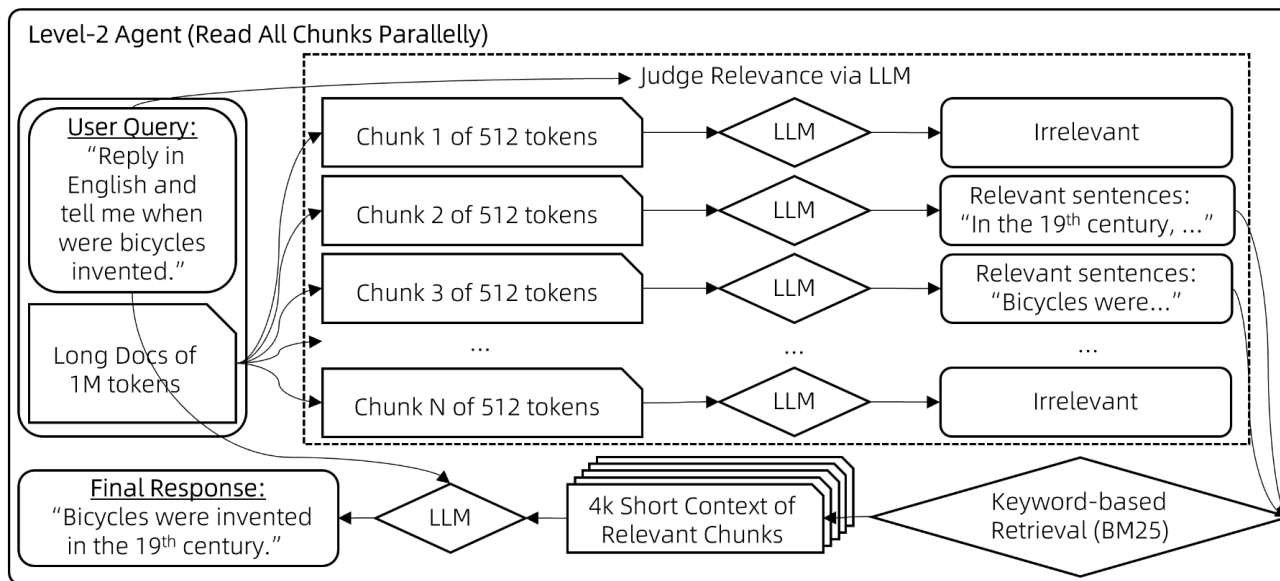
直接RAG检索很快，但常在相关块与用户查询关键词重叠程度不足时失效 => 导致这些相关的块未被检索到。

尽管理论上向量检索可以缓解这一问题，但实际上效果有限。为了解决这个局限，我们采用了一种暴力策略：

步骤1：对于每个512字块，让聊天模型评估其与用户查询的相关性，如果认为不相关则输出"无"，如果相关则输出相关句子。这些块会被并行处理以避免长时间等待。

步骤2：取那些非"无"的输出（即相关句子），用它们作为搜索查询词，通过BM25检索出最相关的块（总的检索结果长度控制在8k上下文限制内）。

步骤3：最后，基于检索到的上下文生成最终答案，这一步骤的实现方式与通常的RAG相同。



级别二：分块阅读（流程图）

搭建RAG (Qwen-Agent)

级别三：逐步推理

在基于文档的问题回答中，一个典型的挑战是多跳推理。

例如，考虑回答问题：“与第五交响曲创作于同一世纪的交通工具有什么？”

模型首先需要确定子问题的答案，“第五交响曲是在哪个世纪创作的？”即19世纪。然后，它才可以意识到包含“自行车于19世纪发明”的信息块实际上与原始问题相关的。

Function Call 智能体或ReAct智能体是经典的解决方案，它们内置了问题分解和逐步推理的能力。因此，我们将前述级别二的智能体（Lv2-智能体）封装为一个工具，由工具调用智能体（Lv3-智能体）调用。

工具调用智能体进行多跳推理的流程如下：

向Lv3-智能体提出一个问题。

```
while (Lv3-智能体无法根据其记忆回答问题) {
```

```
    Lv3-智能体提出一个新的子问题待解答。
```

```
    Lv3-智能体向Lv2-智能体提问这个子问题。
```

```
    将Lv2-智能体的回应添加到Lv3-智能体的记忆中。
```

```
}
```

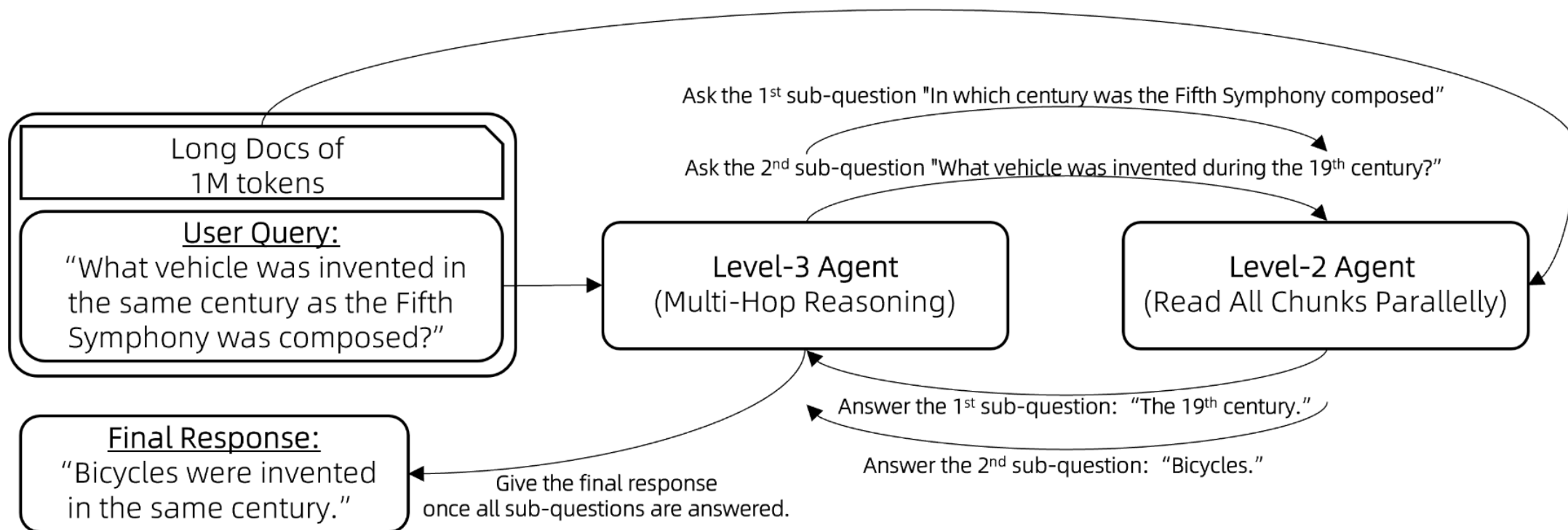
Lv3-智能体提供原始问题的最终答案。

搭建RAG (Qwen-Agent)

例如，Lv3-智能体最初向Lv2-智能体提出子问题：“**贝多芬的第五交响曲是在哪个世纪创作的？**”

收到“**19世纪**”的回复后，Lv3-智能体提出新的子问题：“**19世纪期间发明了什么交通工具？**”

通过整合Lv2-智能体的所有反馈，Lv3-智能体便能够回答原始问题：“与第五交响曲创作于同一世纪的交通工具是什么？”



级别三：逐步推理（流程图）

Qwen-Agent中的检索实现

@qwen-agent-multi-files.py 理解这里的RAG实现过程，调用的qwen_agent工具，先将文档拆分成小块，存放到 workspace中的tools目录下，帮我查找在 qwen_agent中的具体实现代码在哪里

好的，我理解您的问题。您想知道 `qwen-agent` 框架在实现 RAG (检索增强生成) 时，具体是如何处理您传入的文档的，特别是文档是如何被拆分、存储以及具体代码实现的位置。

根据您的提供的 `qwen-agent-multi-files.py` 代码，我们可以定位到关键的入口：

```
bot = Assistant(llm=llm_cfg, ..., files=files)
```

这里的 `files` 参数就是 RAG 知识库的来源。`Assistant` 类在初始化时会接收这些文件，并进行一系列处理，最终构建出一个 `retriever` 对象，用于在运行时进行检索。

您猜测的“将文档拆分成小块，存放到 workspace 中的 tools 目录下”这个流程，非常接近 `qwen-agent` 的实际做法。这个过程由框架在内部自动完成。

为了找到具体的实现代码，我将首先在代码库中搜索 `Assistant` 类的定义，从而找到它处理 `files` 参数的逻辑。

好的，搜索结果已经出来了。

正如我们所料，`qwen_agent/agents/assistant.py` 是核心文件，但它继承了更底层的能力。`Assistant` 类的初始化方法 `__init__` 调用了其父类 `FnCallAgent` 的 `__init__` 方法，并将 `files` 和 `rag_cfg` 参数传递了过去。

真正的RAG处理逻辑并不在 `Assistant` 类本身，而是在它所依赖的 `self.mem` (Memory) 对象中。从 `Assistant` 的 `_run` 和 `_prepend_knowledge_prompt` 方法中可以看到，它通过调用 `self.mem.run()` 来获取知识 (knowledge)。

这说明，处理文档、分块、存储和检索的核心代码位于 `qwen_agent` 的 `Memory` 模块中。

现在，我将深入 `qwen_agent/memory/memory.py` 文件，来查找具体的实现。我将阅读这个文件，重点关注它如何初始化、处理传入的文件、如何进行文本分割 (chunking) 以及使用了哪些检索方法。

通过定位分析，可以看到关键实现在于：`qwen_agent/memory/memory.py`，它加载了两个工具：`retrieval` 和 `doc_parser`。

Qwen-Agent中的检索实现

当用户在初始化 Assistant 对象时传入 files 参数，RAG 流程便被激活。整个过程的核心并非在 Assistant 类中直接实现，而是委托给了其内部的 self.mem（一个 Memory 类的实例）来处理。

Memory 类在初始化时，会默认加载两个关键工具：

- **retrieval**：负责接收用户查询，并从文档中检索最相关的信息。
- **doc_parser**：负责解析不同格式的文档，并将其内容进行切分。

Qwen-Agent中的检索实现

关键步骤1. 文档解析与分块 (Chunking)

这是 RAG 的基础，将大型文档转化为易于检索的小单元。

- 触发点：当 retrieval 工具需要进行信息检索时，它首先需要获取文档内容。
- 核心实现：文档解析与分块的逻辑主要封装在 `qwen_agent/tools/simple_doc_parser.py` 文件中。
- 工作流程：
 1. retrieval 工具调用 doc_parser 工具来处理文件。
 2. doc_parser 会根据文件扩展名（如 .pdf, .docx, .txt 等）自动选择合适的内部解析器。
 3. 选定的解析器读取文件内容，并根据 parser_page_size 参数（可在 rag_cfg 中配置，默认值为 500 字符）将文本切分成多个文本块 (chunks)。
 4. 每一个文本块都被构造成一个标准的 Python 字典，格式通常为 {'page_content': '...', 'metadata': {...}}, 其中 metadata 包含了源文件名、页码等信息。

Qwen-Agent中的检索实现

关键步骤2. 文档块的存储

关于分块后数据的存放位置，qwen-agent 的默认实现有其特殊之处。

- 存储方式：框架并不会将分块后的文档持久化存储到磁盘上的 workspace/tools 目录中。workspace 目录更多是为 code_interpreter 这类需要读写临时文件的工具准备的。
- 工作流程：
 1. doc_parser 解析完所有文档后，会返回一个包含所有文本块的列表（List of Dictionaries）。
 2. 这个列表是 **临时的，并且完全存储在内存中**。
 3. 它的生命周期仅限于当前的这一次请求。当 retrieval 工具需要检索时，它会直接遍历这个内存中的列表来完成操作。

Qwen-Agent中的检索实现

关键步骤 3. 检索 (Retrieval)

这是 RAG 的核心步骤，即根据用户问题找到最相关的知识。

- 触发点：在 Memory 类的 `_run` 方法中，会调用 `self.function_map['retrieval'].call()` 来启动检索过程。
- 核心实现：检索算法的逻辑主要位于 `qwen_agent/tools/retrieval.py` 文件中。
- 工作流程：
 1. retrieval 工具首先通过 `doc_parser` 获取所有文档在内存中的分块列表。
 2. 接着，它会根据 `rag_searchers` 配置项来决定使用哪种搜索方法。默认配置包括：
 - `keyword_search`: 使用经典的 **BM25 算法**，计算用户查询和每个文本块之间的相关度得分。
 - `front_page_search`: 优先检索文档的起始部分。
 3. retrieval 工具执行搜索，对所有文本块进行打分和排序。
 4. 最后，它将得分最高的若干个文本块作为最终的检索结果返回。

Summary (Qwen-Agent中的检索实现)

Qwen-Agent中的检索实现:

- 文档拆分代码: `qwen_agent/tools/simple_doc_parser.py`
- 检索逻辑代码: `qwen_agent/tools/retrieval.py`
- 数据存储位置: 内存中。数据是临时的, 仅为单次查询服务, 不会被持久化到 `workspace` 目录。

通过这种"解析即用"的内存式处理流程, `qwen-agent` 实现了一个轻量级且高效的 RAG 系统, 无需依赖外部的向量数据库或持久化存储。

Version2: 海量文件快速索引 (ES)

Elasticsearch使用

Elasticsearch（简称ES）：

一个开源的高扩展的分布式全文检索引擎，可以免费使用。

由于ES是基于Java开发的，所以在安装时需要先安装Java环境，且Java版本通常需要1.8以上。不过从Elasticsearch 7.0开始，其安装包中包含了一个相匹配的OpenJDK版本，因此可以不单独安装Java

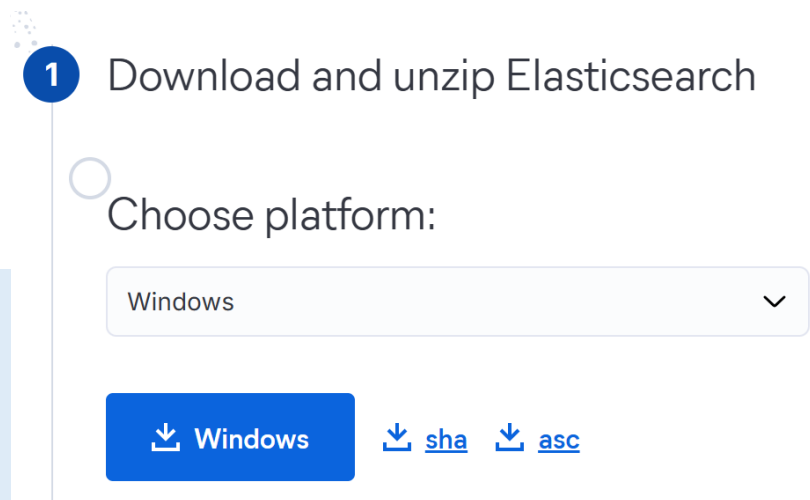
1. 下载Elasticsearch

<https://www.elastic.co/cn/downloads/elasticsearch>

Elasticsearch 是一个 Java 程序，因此系统上必须安装 Java。

检查Java是否安装成功：打开命令行工具(cmd)，

输入 `java -version`，如果能看到版本号 => 说明安装成功



Elasticsearch使用

2. 运行 Elasticsearch

Step1. 运行启动脚本：进入 bin 文件夹，双击运行 elasticsearch.bat 文件。

Step2. 观察启动过程：此时会弹出一个命令行窗口，开始打印启动日志。首次启动会生成一些配置和安全设置，包括一个 elastic 超级用户的密码（你需要将这个密码保存下来 => 后面会用到）

4. ****验证是否成功****：等待启动完成后（日志不再滚动），打开浏览器访问 <https://localhost:9200>。浏览器会提示您输入用户名和密码。输入用户名 elastic 和您刚刚保存的密码。如果能看到一个包含版本信息的 JSON 响应，说明 Elasticsearch 已成功在您的 Windows 上运行。

登录

<https://localhost:9200>

用户名

elastic

密码

.....

登录

取消



```
<  →  ↻  ⚠ 不安全  https://localhost:9200
美观输出 ☐
{
  "name" : "CY",
  "cluster_name" : "elasticsearch",
  "cluster_uuid" : "DWnN3BG1SPmpl-jwJHlgwg",
  "version" : {
    "number" : "9.0.2",
    "build_flavor" : "default",
    "build_type" : "zip",
    "build_hash" : "0a58bc1dc7a4ae5412db66624aab968370bd44ce",
    "build_date" : "2025-05-28T10:06:37.834829258Z",
    "build_snapshot" : false,
    "lucene_version" : "10.1.0",
    "minimum_wire_compatibility_version" : "8.18.0",
    "minimum_index_compatibility_version" : "8.0.0"
  },
  "tagline" : "You Know, for Search"
}
```

说明：关闭 elasticsearch.bat 的运行窗口就会关闭 ES 服务

Elasticsearch使用

Thinking: 如果不知道用户名, 密码怎么办?

```
curl -k https://localhost:9200
```

```
{"error":{"root_cause":[{"type":"security_exception","reason":"missing authentication credentials for REST request [/]","header":{"WWW-Authenticate":["Basic realm=\"security\\", charset=\"UTF-8\\", \"Bearer realm=\"security\\", \"ApiKey\"]}], "type":"security_exception","reason":"missing authentication credentials for REST request [/]","header":{"WWW-Authenticate":["Basic realm=\"security\\", charset=\"UTF-8\\", \"Bearer realm=\"security\\", \"ApiKey\"]}], "status":401}
```



Elasticsearch 返回了一个 401 错误, 表示请求缺少认证凭证, 需要进行身份验证。

重置密码: `bin/elasticsearch-reset-password -u elastic`

```
warning: ignoring JAVA_HOME=D:\\Program Files\\Java\\jdk-22; using bundled JDK
This tool will reset the password of the [elastic] user to an autogenerated value.
The password will be printed in the console.
Please confirm that you would like to continue [y/N]y
```

```
Password for the [elastic] user successfully reset.
New value: euqPcOlHrmW18rtaS-3P
```

Elasticsearch使用

Thinking: 如何在Python中使用ES?

Step1, 安装es工具包

`pip install elasticsearch`

Step2, 编写python-es, 完成文档的索引和检索

docs 文件夹里面有很多平安雇主险的文档, 帮我

1) 连接到ES

2) 创建一个索引

3) 索引文档 (添加这些数据)

4) 执行搜索

`search_query = "工伤保险和雇主险有什么区别?"`

5) 显示搜索结果

编写 .py

好的, 我明白了。我将为您编写一个 Python 脚本, 该脚本会完成您指定的五个步骤:

1. 连接到您本地的 Elasticsearch 服务。
2. 创建一个新的索引 (如果已存在则会重建)。
3. 读取 docs 文件夹下的所有文档, 并将它们的内容添加到 ES 索引中。
4. 使用您提供的查询语句 "工伤保险和雇主险有什么区别?" 执行搜索。
5. 在命令行中打印出搜索结果。

首先, 为了能读取 docs 文件夹下的各种文档 (特别是 PDF 文件), 我们需要安装一个额外的库 `pypdf`。

我将为您生成安装这个库的命令。

```
pip install pypdf
```

Output

```
index_and_search_docs.py +124 ✓  
  
import os  
from elasticsearch import Elasticsearch  
from pypdf import PdfReader  
import warnings  
  
# 忽略来自 pypdf 的特定用户警告  
warnings.filterwarnings("ignore", category=UserWarning, module='pypdf')  
  
def index_and_search_documents():  
    """  
    连接到ES, 索引docs文件夹下的文档, 并执行搜索。  
    """  
    # --- 1. 连接到 Elasticsearch ---
```


Elasticsearch使用

index_and_search_docs.py > index_and_search_documents

```
1 import os
2 from elasticsearch import Elasticsearch
3 from pypdf import PdfReader
4 import warnings
5
6 # 忽略来自 pypdf 的特定用户警告
7 warnings.filterwarnings("ignore", category=UserWarning, module='pypdf')
8
9
```

```
10 def index_and_search_documents():
11     """
12     连接到ES, 索引docs文件夹下的文档, 并执行搜索。
13     """
14     # --- 1. 连接到 Elasticsearch ---
15     # 请确保将 "YOUR_ELASTIC_PASSWORD" 替换为您的真实密码
16     try:
17         client = Elasticsearch(
18             "https://localhost:9200", # 1. 使用 https
19             basic_auth=("elastic", "euqPcOlHrmW18rtaS-3P"),
20             verify_certs=False # 2. 忽略SSL证书验证 (等同于 curl -k)
21         )
22         if not client.ping():
23             print("无法连接到 Elasticsearch, 请检查服务是否正在运行以及密码")
24         return
25     print("成功连接到 Elasticsearch!")
```

```
- 成功索引文档: 平安境内紧急医疗救援服务条款.pdf (ID: 11)
incorrect startxref pointer(1)
D:\AppData\Local\Programs\Python\Python311\Lib\site-packages\urllib3\connectionpool.py:1061: InsecureRequestWarning: Unverified HTTPS request is being made to host 'localhost'. Adding certificate verification is strongly advised. See: https://urllib3.readthedocs.io/en/1.26.x/advanced-usage.html#ssl-warnings
warnings.warn(
- 成功索引文档: 平安附加疾病身故保险条款.pdf (ID: 12)
```

```
所有文档索引完成!
D:\AppData\Local\Programs\Python\Python311\Lib\site-packages\urllib3\connectionpool.py:1061: InsecureRequestWarning: Unverified HTTPS request is being made to host 'localhost'. Adding certificate verification is strongly advised. See: https://urllib3.readthedocs.io/en/1.26.x/advanced-usage.html#ssl-warnings
warnings.warn(
```

```
正在执行搜索, 查询语句: '工伤保险和雇主险有什么区别?'
D:\AppData\Local\Programs\Python\Python311\Lib\site-packages\urllib3\connectionpool.py:1061: InsecureRequestWarning: Unverified HTTPS request is being made to host 'localhost'. Adding certificate verification is strongly advised. See: https://urllib3.readthedocs.io/en/1.26.x/advanced-usage.html#ssl-warnings
warnings.warn(
```

--- 搜索结果 ---

```
--- 结果 1 ---
来源文件: 2-雇主责任险.txt
相关度得分: 10.10
内容预览: 【雇主责任险】 Q1 雇员意外事故给企业造成的损失有多大? 甚至可能让一家公司倒闭!! 员工出外勤不幸遇到车祸被撞高位截瘫 公司赔偿近一百万 抽干公司流动资金 无力维持生产 走破产程序... Q2 雇主责任险的保障范围和其他亮点
1. 保障 只要有用工需求的单位, 都可以购买雇主责任险。 2. 保障范围: 死亡赔偿金 伤残赔偿金 医疗费用 误工费 (c款不赔) 法律诉讼费 (c款不赔) 3. 其他亮点...
```

```
--- 结果 2 ---
来源文件: 4-雇主安心保.txt
相关度得分: 9.16
内容预览: 【雇主安心保】 Q1 什么是雇主安心保? 工伤保险也能很简单 一个小老板的心声..... • 工伤保险待遇纠纷数占社会保险纠纷71.90% • 工伤事故处理不好, 很有可能让一家小型企业倒闭! 我最大的希望就是手下这群工人, 每天都能平平安安的。无论是谁出了事故, 都赔不起啊! 雇主安心保, 真正从雇主权益角度出发! • 投保免核、灵活定制, 你担心的风险, 全部由它买单! Q2 雇主安心保的赔偿范围 雇主安心保...
```

使用ES可以快速得到检索结果

索引管理

```
index_name = "pingan_employer_insurance"
if client.indices.exists(index=index_name):
    print(f"索引 '{index_name}' 已存在, 正在删除旧索引...")
    client.indices.delete(index=index_name)

print(f"正在创建新索引 '{index_name}'...")
client.indices.create(index=index_name)
```

定义索引名: `index_name = "pingan_employer_insurance"`

删除旧索引 (如果存在):

- `client.indices.exists(...)`: 检查名为 `pingan_employer_insurance` 的索引是否已经存在。
- `client.indices.delete(...)`: 如果存在, 则将其删除。

目的: 确保每次运行脚本都是在一个全新的、干净的索引上进行操作, 避免了旧数据的干扰。

创建新索引: `client.indices.create(index=index_name)`, 创建一个新的、空的索引。

读取、解析并索引文档

```
docs_folder = 'docs'
...
print("\n正在从 'docs' 文件夹读取并索引文档...")
doc_id_counter = 1
for filename in os.listdir(docs_folder):
    ...
    if content.strip():
        client.index(
            index=index_name,
            id=doc_id_counter,
            document={
                "file_name": filename,
                "content": content
            }
        )
        print(f" - 成功索引文档: {filename} (ID: {doc_id_counter})")
        doc_id_counter += 1
# ...
# 强制刷新索引, 确保数据立即可搜
client.indices.refresh(index=index_name)
```

遍历文件: 使用 `os.listdir()` 遍历 docs 文件夹下的所有项目。

文件类型判断:

- 通过 `filename.lower().endswith('.pdf')` 和 `.txt` 来判断文件类型。对于不支持的类型, 会打印一条消息并跳过。

内容提取:

- ****PDF 文件****: 使用 `PdfReader` 打开 PDF, 逐页提取文本。
- ****TXT 文件****: 直接读取整个文件的内容。

发送到 Elasticsearch:

- **`client.index(...)`**: 对于提取到内容的文件, 将其发送到 ES。
- `index=index_name`: 指定要存入哪个索引。
- `id=doc_id_counter`: 为每条记录分配一个从 1 开始递增的ID。
- `document={...}`: 这是要存入的实际数据, 一个包含文件名和提取内容的 JSON 对象。

刷新索引: `client.indices.refresh(index=index_name)`, 这是一个重要操作。在 ES 中, 新写入的数据需要"刷新"后才能被搜索到。

执行搜索

```
search_query = "工伤保险和雇主险有什么区别？"
print(f"\n正在执行搜索，查询语句: '{search_query}'")
response = client.search(
    index=index_name,
    query={
        "match": {
            "content": {
                "query": search_query,
                "operator": "and" # 使用 and 操作符，要求所有词都
出现，更精确
            }
        }
    },
    size=3 # 返回最相关的3个结果
)
```

定义查询: `search_query = "工伤保险和雇主险有什么区别？"`

构建搜索请求:

- `client.search(...)`
- `query={"match": {"content": ...}}`: 使用了 Elasticsearch 中最常见的 `match` 查询。它会对查询字符串进行分词，然后在 `content` 字段中查找包含这些分词的文档。
- `"operator": "and"`: 要求文档必须**同时包含**查询分词后的大部分重要词语（如"工伤保险"和"雇主险"），这让搜索结果更精确。
- `size=3`: 指定最多返回 3 条最相关的结果。

显示搜索结果

```
print("\n--- 搜索结果 ---")
hits = response['hits']['hits']
if not hits:
    print("没有找到匹配的文档。")
else:
    for i, hit in enumerate(hits):
        print(f"\n--- 结果 {i+1} ---")
        print(f"来源文件: {hit['_source']['file_name']}")
        print(f"相关度得分: {hit['_score']:.2f}")
        content_preview =
hit['_source']['content'].strip().replace('\n', ' ')
        print(f"内容预览: {content_preview[:200]}...")
```

解析响应: ES 的返回结果是一个复杂的 JSON 结构, 脚本通过 `response['hits']['hits']` 来提取包含所有匹配文档的列表。

遍历结果: 循环遍历每个匹配项 (hit)。

打印信息:

- `hit['_source']['file_name']`: 打印文档的来源文件名。
- `hit['_score']`: 打印由 ES 计算出的相关度得分。
- `hit['_source']['content'][:200]`: 打印文档内容的前 200 个字符

Elasticsearch使用

Thinking: 这个搜索结果是以文档为单位么? 相关度得分是如何计算的?

是的, 因为索引逻辑是: 遍历 docs 文件夹, 它将每一个文件 (例如 2-雇主责任险.txt) 的全部内容读取出来, 作为一个整体, 存入了 Elasticsearch 的一个“文档 (Document)”中。

当执行搜索时, Elasticsearch 会去检查您的查询语句 (“工伤保险和雇主险有什么区别?”) 和哪条记录的 content 字段最匹配。如果找到了, 它返回的就是那条完整的记录。

Thinking: 如果希望结果更精确, 比如直接返回最相关的段落, 如何实现?

不再将整个文件存为一个ES文档, 而是先把文件切分成段落, 然后将段落作为一个独立的ES文档存入索引中, 并附上它所属的源文件名

=> 搜索结果就能精确到段落级别。

BM25得分计算

Thinking: ES的相关度得分是如何计算的?

这个得分是 Elasticsearch 的核心功能，使用的BM25算法 (Best Match 25)

可以将其理解为一个综合打分机制，主要考虑以下三个因素：

- 词频 (Term Frequency - TF)

含义：查询词（如“工伤保险”、“雇主险”）在文档内容中出现了多少次？

原则：出现次数越多，得分越高。一个反复提到“雇主险”的文档，显然比只提一次的文档更相关。

- 逆文档频率 (Inverse Document Frequency - IDF)

含义：您的查询词在所有文档中是常见词还是稀有词？

原则：词越稀有，权重越高。例如，在保险文档库中，“保险”这个词几乎每个文档都有，所以它的权重很低；而“高空坠物责任”这种词可能只在少数文档中出现，它的权重就很高。如果您的查询命中了稀有词，得分会大幅提升。

BM25得分计算

- 字段长度归一化 (Field-Length Norm)

含义：匹配到的查询词所在字段的长度。

原则：字段越短，得分越高。

在一个很短的文档中找到“雇主险”，比在一个五千字的文档中找到“雇主险”，更能凸显这个词的重要性。

Elasticsearch的搜索过程

当我们搜索“工伤保险和雇主险有什么区别？”时，Elasticsearch 会：

1. 分析查询，提取出“工伤保险”、“雇主险”、“区别”等关键词。
2. 遍历索引中的每一个文档。
3. 对每个文档，综合 **TF**、**IDF** 和**字段长度** 三大因素，为每个关键词计算一个分数。
4. 将所有关键词的分数合并，得出一个最终的 `_score`。

这就是为什么 `_score` 是一个浮点数，它精确地反映了文档与查询之间复杂的关联程度

Elasticsearch 可以处理的文档数量取决于多个因素，包括集群的硬件配置、索引配置、分片数量以及数据模型等。单个索引中文档数量的上限为2,147,483,519。对于上亿级别的文档，Elasticsearch 也能够处理，但性能可能会受到硬件资源和配置的影响。

在性能方面，Elasticsearch 单机的检索性能在0.005~5.6秒之间，可满足95%+的业务场景

Qwen-Agent + ES

将初版代码保存到 ai-bot-1.py

找到Python环境中的qwen_agent安装包，比如 C:\Users\cheny\AppData\Roaming\Python\Python311\site-packages\qwen_agent，复制到当前目录下 => 方便后续调整，不影响到原有的qwen-agent工具

@ai_bot-1.py 这里的files是多个文件，会通过Assistant中的 memory.py (qwen_agent/memory/memory.py) 加载两个工具：retrieval 和 doc_parser。

retrieval：负责接收用户查询，并从文档中检索最相关的信息。

doc_parser：负责解析不同格式的文档，并将其内容进行切分。

===

文档拆分代码: qwen_agent/tools/simple_doc_parser.py

检索逻辑代码: qwen_agent/tools/retrieval.py

===

我想改成用 elasticsearch来进行文档片段chunks的索引和管理，提升检索性能。帮我扩展 qwen_agent文件夹中的代码，新增该功能实现，放到 qwen_agent对应的文件夹下面即可。

然后让 Assistant中的memory，有一个选项是可以用es来实现检索和管理

如果要改写 ai_bot-1.py，可以写入到新的 ai_bot-3.py

Qwen-Agent + ES

Let's start with **Step 1**: Creating the core `elasticsearch_searcher.py` file.

```
elasticsearch_searcher.py +126 ✓  
  
# qwen_agent/searcher/elasticsearch_searcher.py  
import os  
from typing import List, Dict  
from elasticsearch import Elasticsearch, helpers  
  
# 忽略不必要的警告  
import warnings  
warnings.filterwarnings("ignore", category=FutureWarning)  
try:  
    from elasticsearch.exceptions import ConnectionError, NotFoundError  
except ImportError:  
    from urllib3.exceptions import ReadTimeoutError as ConnectionError  
    from elasticsearch.exceptions import NotFoundError
```

在 qwen_agent 工具箱中，会创建一个 searcher 的文件夹，放入了 elasticsearch_searcher.py

```
es_retrieval.py +68 ✓  
  
# qwen_agent/tools/es_retrieval.py  
import json  
from typing import Dict, List  
from qwen_agent.tools.base import BaseTool  
from qwen_agent.tools.doc_parser import DocParser  
from qwen_agent.searcher.elasticsearch_searcher import ElasticsearchSe  
  
class ESRetrievalTool(BaseTool):  
    """  
    一个使用 Elasticsearch 作为后端的检索工具。  
    它协调文档解析、分块、索引和搜索的整个流程。  
    """  
    name = 'retrieval' # 保持名称为 'retrieval' 以便在 Memory 中进行替换  
    description = '从 Elasticsearch 索引的文档中检索与用户查询相关的内容。'
```

在 qwen_agent 工具箱中的 tools 文件夹下面，会放入一个 es_retrieval.py

Qwen-Agent + ES

```
memory.py +26 -7 ✓

from qwen_agent.tools import BaseTool
from qwen_agent.tools.simple_doc_parser import PARSER_SUPPORTED_FILE_T
from qwen_agent.utils.utils import extract_files_from_messages, extrac
from qwen_agent.tools.es_retrieval import ESRetrievalTool

class Memory(Agent):
    28 hidden lines
    if not llm:
        # There is no suitable model available for keygen
        self.rag_keygen_strategy = 'none'

    # --- 新增: 根据配置选择 RAG 后端 ---
    self.rag_backend = self.cfg.get('rag_backend', 'default') # 默
```

调整了原有 memory.py 的实现逻辑，对应文件
qwen_agent/memory/memory.py

```
ai_bot-3.py +76 ✓

# ai_bot-3.py
import os
import asyncio
from typing import Optional
from qwen_agent.agents import Assistant
from qwen_agent.gui import WebUI

def init_agent_service():
    """初始化具备 Elasticsearch RAG 能力的助手服务"""

    # 步骤 1: LLM 配置
    llm_cfg = {
        'model': 'qwen-max',
        'model_server': 'dashscope',
```

创建了 ai_bot-3.py 实现了将files进行 es索引和管理
的功能

Qwen-Agent + ES

Thinking: 运行 ai-bot-3.py 的时候报错, 如何进行修复?

等待了很长的时间, 最后还是没有检索到。帮我写一个额外的测试文件, 就是单独测试下 es 检索回复, 比如针对这个用户输入的问题: 介绍下雇主责任险

好的, 我完全理解您的需求。经过多次调试后, 应用仍然没有按预期工作, 这确实令人沮丧。创建一个独立的、最小化的测试脚本来专门测试 Elasticsearch 的索引和检索功能, 是目前最直接、最有效的调试方法。这可以帮助我们彻底排除干扰, 看看问题究竟是在我们自建的 ES 工具链中, 还是在 `qwen-agent` 框架与工具的交互中。

我将为您创建一个新的测试文件 `test_es_retrieval.py`。这个脚本会:

1. 直接实例化我们创建的 `ESRetrievalTool`。
2. 模拟框架的行为, 调用该工具的 `call` 方法。
3. 传入您指定的查询 "介绍下雇主责任险" 和 `docs` 目录下的所有文件。
4. 清晰地打印出整个过程的日志和最终的检索结果。

```
test_es_retrieval.py +95 ✓  
  
# test_es_retrieval.py  
import os  
import json  
import logging  
from qwen_agent.tools.es_retrieval import ESRetrievalTool  
  
# 设置日志, 方便观察详细过程  
logging.basicConfig(level=logging.INFO, format='%(asctime)s - %(message)s')  
  
def test_retrieval():  
    """  
    一个独立的测试函数, 用于验证 ESRetrievalTool 的索引和检索功能。  
    """  
    print("--- 开始独立测试 Elasticsearch 检索工具 ---")
```

Qwen-Agent + ES

```
9 def test_retrieval():
10     """独立测试 Elasticsearch 检索工具的功能。"""
11     print("--- 开始独立测试 Elasticsearch 检索工具 ---")
12
13     # --- 步骤 1: 准备输入参数 ---
14     query = "介绍下雇主责任险"
15     docs_folder = 'docs'
16     files_to_process = [os.path.join(docs_folder, f) for f in os.listdir(docs_folder)]
17
18     print("\n--- 步骤 1: 准备输入参数 ---")
19     print(f"查询语句: {query}")
20     print(f"待处理文件数: {len(files_to_process)}")
21
22     # 准备工具的配置
23     # 确保 'es' 配置部分与您的 Elasticsearch 实例匹配
24     tool_cfg = {
25         'es': {
26             'host': 'https://localhost', # 或者您的 ES 主机
27             'port': 9200,
28             'index_name': 'my_insurance_docs_index_test', # 使用专用的
29             # 如果您的 Elasticsearch 开启了认证, 请取消下面的注释并填写正
30             'user': 'elastic',
31             'password': 'euqPc0lHrmW18rtaS-3P',
32         }
33     }
34
35     # 实例化工具
36     retrieval_tool = ESRetrievalTool(cfg=tool_cfg)
```

```
2025-06-22 20:31:47,619 - qwen_agent.searcher.elasticsearch_searcher - INFO - 在 Elasticsearch 中发现 13 个已存在的文档块。
2025-06-22 20:31:47,619 - qwen_agent.searcher.elasticsearch_searcher - INFO - 筛选出 0 个新块需要索引。
2025-06-22 20:31:47,619 - qwen_agent.searcher.elasticsearch_searcher - INFO - 所有文件内容均已在 Elasticsearch 中建立索引, 无需更新。
2025-06-22 20:31:47,620 - qwen_agent.searcher.elasticsearch_searcher - INFO - 正在使用查询语句在 Elasticsearch 中搜索: '介绍下雇主责任险'
D:\AppData\Local\Programs\Python\Python311\Lib\site-packages\urllib3\connectionpool.py:1061: InsecureRequestWarning: Unverified HTTPS request is being made to host 'localhost'. Adding certificate verification is strongly advised. See: https://urllib3.readthedocs.io/en/1.26.x/advanced-usage.html#ssl-warnings
  warnings.warn(
2025-06-22 20:31:47,633 - elastic_transport.transport - INFO - POST https://localhost:9200/my_insurance_docs_index_test/_search [status:200 duration:0.013s]
2025-06-22 20:31:47,634 - qwen_agent.searcher.elasticsearch_searcher - INFO - 搜索完成, 找到 3 个结果。
```

--- 步骤 3: 解析并展示检索结果 ---
成功从 Elasticsearch 检索到 3 条相关内容:

--- 结果 1 ---
来源: docs\平安境内紧急医疗救援服务条款.pdf
得分: 7.47124
内容预览: 中国平安财产保险股份有限公司
平安境内紧急医疗救援服务条款

总则

第一条 本救援服务条款须附加于意外伤害保险合同（以下简称“主保险合同”）。主保险合同所附条款、投保单、保险单、保险凭证以及批单等，凡与服务条款相关者，均为本服务条款的构成部分。凡涉及本服务的约定，均应采用书面形式。
若主保险合同与本服务条款互有冲突，则以本服务条款为准。本服务条款未尽事宜，以主保险合同的条款规定为准。
救援服务内容

第二条 在主保险合同保险期间内，如果被保险人在中华人民共和国境内（不含港澳台地区，以下简称“境内”）且非其境内常住地（以下简称“非常住地”）遭受主保险合同约定的...

--- 结果 2 ---
来源: docs\3-平安企业团体综合意外险.txt
得分: 4.624165
内容预览: 【平安企业团体综合意外险】

通过多次调试，跑通了 test_es_retrieval.py

Qwen-Agent + ES

TO DO: 确保实现的 `es_retrieval.py` 和 `retrieval.py` 的代码逻辑一致

@`retrieval.py` 是不是把所有匹配的都返回了？ @`es_retrieval.py` 匹配上的返回了多少？

这是一个非常好的问题，它触及了两个检索工具在返回结果数量上的核心差异。

我来为您分别解答：

1. @`retrieval.py` (默认内存检索) 返回多少结果？

答案：默认情况下，它不会返回所有匹配的结果，而是有一个上限。

这个工具的设计是可组合的，它将实际的搜索工作委托给一个或多个“子检索器”。默认情况下，它使用的是 `HybridSearch` (混合搜索)，这个混合搜索又调用了 `keyword_search` (关键词搜索) 和 `front_page_search` (首页搜索) 等。

问题的关键在于 `keyword_search` 这个子检索器。我需要查看它的源代码来确定限制。

我先来查找 `keyword_search.py` 这个文件。

🔍 Searched files

好的，文件找到了。现在我来读取 `qwen_agent/tools/search_h_tools/keyword_search.py` 的内容，看看它是如何限制返回结果数量的。

结论

1. @`retrieval.py` (默认): 不，它不返回所有匹配的结果。它会根据一个 `max_ref_token` 的上限（默认为 20000）来截取最相关的文档块，直到累积的 token 总数接近这个上限为止。它旨在返回一个能在 LLM 上下文中容纳的、最精华的部分。
2. @`es_retrieval.py` (我们的): 它默认返回 3 个最匹配的结果。这个数量是硬编码在 `ElasticsearchSearcher` 中的。

Qwen-Agent + ES

@es_retrieval.py 返回的上限能否参考 @retrieval.py 这里的max_ref_token 逻辑是一样的

第一步：改造索引逻辑，存入 Token 数量

我将修改 `qwen_agent/searcher/elasticsearch_searcher.py`，在 `index_files` 方法中，将 `chunk['token']` 添加到要索引的 `_source` 中。

elasticsearch_searcher.py +3 -1 ✓

```
        "_id": chunk['id'],
        "_source": {
            "content": chunk['content'],
            "source": chunk['metadata']['source'],
            "token": chunk.get('token', 0)
        }
    } for chunk in new_chunks]

except Exception as e:
    logger.error(f"Elasticsearch 搜索失败: {e}")
    return []
```

第二步：改造搜索逻辑，使用 max_ref_token

接下来，我将修改同一个文件 (`elasticsearch_searcher.py`) 中的 `search` 方法。这个改动会比较大：

- 函数签名将从 `search(self, query: str, k: int = 3)` 变为 `search(self, query: str, max_ref_token: int)`。
- 它会向 ES 请求一批较多的候选结果（比如 100 个）。
- 然后，它会在本地循环这些结果，累加每个块的 `token` 数量，直到达到 `max_ref_token` 的上限，最后返回被选中的结果。

elasticsearch_searcher.py +24 -5 ✓

```
def search(self, query: str, k: int = 3) -> list:
    """在 Elasticsearch 中执行搜索并返回前 k 个结果"""
    def search(self, query: str, max_ref_token: int) -> list:
        """在 Elasticsearch 中执行搜索，并根据 max_ref_token 限制返回结果"""
        if not self.client:
            logger.error("Elasticsearch 客户端不可用")
            return []

        logger.info(f"正在使用查询语句在 Elasticsearch 中搜索: {query}")
```

第三步：改造 ESRetrievalTool，传递 max_ref_token

现在，我们只需要让 `ESRetrievalTool` 能够读取 `max_ref_token` 配置，并在调用 `search` 方法时把它传递过去。

这将为两小步：

- 在 `__init__` 中，像默认的 `Retrieval` 工具一样，从配置中读取 `max_ref_token`。
- 在 `call` 方法中，将 `self.max_ref_token` 传递给 `self.searcher.search()`。

我现在就对 `qwen_agent/tools/es_retrieval.py` 进行修改。

es_retrieval.py +4 -1 ✓

```
from qwen_agent.tools.doc_parser import DocParser
from qwen_agent.utils.utils import print_traceback
from qwen_agent.searcher.elasticsearch_searcher import ElasticsearchSearcher
from qwen_agent.settings import DEFAULT_MAX_REF_TOKEN

class ESRetrievalTool(BaseTool):
    """
    17 hidden lines
    def __init__(self, cfg: dict = None):
        super().__init__(cfg)
        self.cfg = cfg or {}
        self.max_ref_token = self.cfg.get('max_ref_token', DEFAULT_MAX_REF_TOKEN)
        # 初始化 ES 搜索器，它内部会管理文档解析
        self.searcher = ElasticsearchSearcher(cfg=self.cfg)
```


Qwen-Agent + ES

介绍下雇主责任险

介绍下雇主责任险



user

雇主责任险是一种为企业提供风险保障的保险产品，主要针对企业员工在工作期间因意外事故或职业病导致的伤残或死亡。以下是关于雇主责任险的一些关键点：

1. 保障范围：

- 死亡赔偿金
- 伤残赔偿金
- 医疗费用
- 误工费用（某些版本不包括）
- 法律诉讼费（某些版本不包括）

2. 其他亮点：

- 雇主责任转嫁给保险公司：**从雇主权益角度出发，帮助企业转移对员工的风险责任。
- 降低企业用工风险：**通过保险机制减轻企业在面临员工伤害时的经济负担。
- 减少工伤纠纷：**有助于更平稳地处理与员工之间的工伤争议。
- 提高员工保障福利：**为员工提供额外的安全感和保障。
- 税前列支：**相关保费可以在计算企业所得税前扣除。

3. **案例说明：**有工厂老板为其员工购买人身意外伤害保险，但当发生员工生产过程中不幸身亡的情况时，家属仍然向雇主索赔高额赔偿。这表明单独的人身意外伤害保险不足以完全覆盖雇主的责任风险，因此需要专门的雇主责任险来进一步保护企业免受此类损失的影响。

4. **理赔流程：**可以通过线上或线下两种方式进行理赔申请。线上流程主要包括登录平安企业宝APP、上传所需资料等步骤；而线下则需拨打95511进行报案，并按指示准备相关材料提交给保险公司以完成理赔过程。

综上所述，雇主责任险是企业为自身及其员工提供了一种重要的安全保障措施，尤其适合那些希望有效管理劳动风险的企业使用。



Qwen-Agent + ES

雇主责任险和工伤保险有什么主要区别？

雇主责任险和工伤保险有什么主要区别？



user



雇主责任险与工伤保险的主要区别体现在以下几个方面：

1. 法律依据：

- 工伤保险是根据国家法律法规要求必须为员工投保的，具有强制性。
- 雇主责任险则属于商业保险范畴，由企业自主选择是否购买。

2. 保障范围：

- 工伤保险基金可以承担大部分的工伤保险待遇，但仍有部分需要用人单位自身承担，例如停工留薪期间的工资福利待遇（误工费），以及五到十级伤残的一次性伤残就业补助金。
- 雇主责任险除了涵盖工伤保险的部分责任外，还可以承保职业性疾病、误工费用等，并且理赔程序相对简单快捷。

3. 对企业的额外价值：

- 企业申领工伤保险金的次数可能反映该单位的劳动安全状况，从而可能招致相应的行政处罚。而雇主责任险赔偿与劳动安全记录无关。
- 投保雇主责任险有助于降低企业的用工风险，减少因工伤导致的纠纷，并可作为企业对员工的一种福利形式存在。

综上所述，虽然工伤保险能够提供基本的工伤保障，但为了更全面地覆盖潜在风险并减轻企业负担，许多企业会选择同时投保雇主责任险来补充工伤保险未能覆盖的部分。



Version3: 添加向量检索 功能

Qwen3-embedding使用

Qwen3-Embedding是阿里巴巴推出的开源文本嵌入模型，基于Qwen3架构优化，支持多语言、长文本和高精度语义理解，适用于检索、排序等任务。

1. 核心能力：

- **多语言支持**：覆盖119种语言，在MTEB多语言榜单排名第一（70.58分）。
- **长文本处理**：支持32K上下文，适合文档分析等场景。
- **灵活维度**：可输出32-4096维向量，适配不同计算需求。

2. 应用场景：

包括跨语言搜索、代码检索、金融文档分类等，尤其适合需要高精度语义匹配的任务。

3. 开源与部署

提供0.6B/4B/8B版本，已在Hugging Face和ModelScope开源，支持阿里云API快速集成。

Qwen3-embedding使用

```
#text-embedding-v4: 属于Qwen3-Embedding系列

import os

from openai import OpenAI

client = OpenAI(
    api_key=os.getenv("DASHSCOPE_API_KEY"),
    base_url="https://dashscope.aliyuncs.com/compatible-mode/v1"
)

completion = client.embeddings.create(
    model="text-embedding-v4",
    input='衣服的质量杠杠的，很漂亮，不枉我等了这么久啊，喜欢，以后还来这里买',
    dimensions=1024, # 指定向量维度
    encoding_format="float"
)

print(completion.model_dump_json())
```

```
{
  "data": [
    {
      "embedding": [
        0.00954423751682043,
        -0.11166470497846603,
        0.0002610872033983469,
        -0.04448245093226433,
        0.018730206415057182,
        -0.013019428588449955,
        0.015362495556473732,
        -0.0032817272003740072,
        -0.0013390234671533108,
        0.12496358901262283,
        0.04709063470363617,
        -0.03614199161529541,
        0.027127988636493683,
        -0.03147018700838089,
        0.0708509162068367,
        -0.07320114970207214,
        -0.038635529577732086,
        -0.08036649227142334,
        -0.04929756000638008,
        -0.03321853280067444,
        0.023817600682377815,
        0.05574636906385422,
        0.028919324278831482,
        -0.0031061763875186443,
        -0.027844524011015892,
        0.03485222905874252,
        -0.01501855906099081,
        -0.05649156495928764,
        -0.008290302008390427,
        0.06265375763177872,
        -0.024706102907657623,
        0.0066888476721942425,
        -0.042590800672769547,
        -0.00920030102133751,
        0.04737725108861923,
        0.019876662641763687,
        -0.016265328973531723,
        0.029979795217514038,
        0
      ]
    }
  ],
  "index": 0,
  "object": "embedding",
  "model": "text-embedding-v4",
  "object": "list",
  "usage": {
    "prompt_tokens": 23,
    "total_tokens": 23
  },
  "id": "d5a4fc86-bb71-9bc9-9858-9d382e532117"
}
```

Qwen3-embedding使用

@index_and_search_docs.py 我想改成基于 embedding向量的召回，这里可以用text-embedding-v4 参考

@qwen3-embedding.py

帮我写入到 index_and_search_docs-embedding.py

```
...ch_docs-embedding.py (new) +170 •
import os
import warnings
from elasticsearch import Elasticsearch
from pypdf import PdfReader
from openai import OpenAI

# 忽略来自 pypdf 的特定用户警告
warnings.filterwarnings("ignore", category=UserWarn

def get_embedding(text: str, client: OpenAI) -> lis
    """使用 Dashscope 的 text-embedding-v4 模型为文本
    try:
        # 确保文本不为空
```



```
1 import os
2 import warnings
3 from elasticsearch import Elasticsearch
4 from pypdf import PdfReader
5 from openai import OpenAI
6 warnings.filterwarnings("ignore")
7
8
9 def get_embedding(text: str, client: OpenAI) -> list:
10     """使用 Dashscope 的 text-embedding-v4 模型为文本生成向量。"""
11     try:
12         # 确保文本不为空
13         if not text.strip():
14             return []
15
16         response = client.embeddings.create(
17             model="text-embedding-v4",
18             input=text,
19             dimensions=1024, # Qwen3-Embedding 系列支持的维度
20             encoding_format="float"
21         )
22         return response.data[0].embedding
23     except Exception as e:
24         print(f" - 获取 embedding 时出错: {e}")
25         return []
26
27
28 def index_and_search_documents_with_embedding():
```

Qwen3-embedding使用

```
成功连接到 Elasticsearch!
正在创建新索引 'pingan_employer_insurance_embedding'...

正在从 'docs' 文件夹读取、生成向量并索引文档...
- 成功索引文档: 1-平安商业综合责任保险 (亚马逊).txt (ID: 1)
- 成功索引文档: 2-雇主责任险.txt (ID: 2)
- 成功索引文档: 3-平安企业团体综合意外险.txt (ID: 3)
- 成功索引文档: 4-雇主安心保.txt (ID: 4)
- 成功索引文档: 5-施工保.txt (ID: 5)
- 成功索引文档: 6-财产一切险.txt (ID: 6)
- 成功索引文档: 7-平安装修保.txt (ID: 7)
- 成功索引文档: 平安产险交通出行意外伤害保险 (互联网版) 产品说明.pdf (ID: 8)
- 成功索引文档: 平安产险交通工具意外伤害保险 (互联网版) 条款.pdf (ID: 9)
- 获取 embedding 时出错: Error code: 400 - {'error': {'code': 'InvalidParameter', 'param': None, 'message':
'Range of input length should be [1, 8192]', 'type': 'InvalidParameter'}, 'id': '495e534a-86e8-9e18-914a-4d220acba0fd', 'request_id': '495e534a-86e8-9e18-914a-4d220acba0fd'}
- 跳过无法生成向量的文档: 平安企业团体综合意外险 (互联网版)适用条款.pdf
- 跳过空文件: 平安商业综合责任保险 (亚马逊).pdf
- 成功索引文档: 平安境内紧急医疗救援服务条款.pdf (ID: 10)
incorrect startxref pointer(1)
- 成功索引文档: 平安附加疾病身故保险条款.pdf (ID: 11)

正在执行向量搜索，查询语句: '工伤保险和雇主险有什么区别?'

--- 向量搜索结果 ---

--- 结果 1 ---
来源文件: 2-雇主责任险.txt
相关度得分 (cosine similarity): 0.8491
```

运行报错，因为 文档长度超过了 8192。这里可以引入 chunk_text 函数：增加了文本分块函数，它可将任意长度的文本，切分成长度为 4000 字符、并有 200 字符重叠的文本块列表。

Qwen3-embedding使用

```
30 def chunk_text(text: str, chunk_size: int = 4000, overlap: int = 200)
31     """将长文本切分成带有重叠的块。"""
32     if len(text) <= chunk_size:
33         return [text]
34
35     chunks = []
36     start = 0
37     while start < len(text):
38         end = start + chunk_size
39         chunks.append(text[start:end])
40         start += chunk_size - overlap
41     return chunks
42
43
44 def index_and_search_documents_with_embedding():
45     """
46     连接到ES,使用 embedding 对文档进行索引,并执行向量搜索。
47     """
48     # --- 1. 初始化客户端 ---
49     # a. Elasticsearch 客户端
50     try:
51         es_client = Elasticsearch(
52             "https://localhost:9200",
53             basic_auth=("elastic", "euqPcOlHrmW18rtaS-3P"),
54             verify_certs=False
55         )
56         if not es_client.ping():
```

成功连接到 Elasticsearch!

索引 'pingan_employer_insurance_embedding' 已存在,正在删除旧索引...
正在创建新索引 'pingan_employer_insurance_embedding'...

正在从 'docs' 文件夹读取、分块、生成向量并索引文档...

- 文件 '1-平安商业综合责任保险(亚马逊).txt'被切分成 1 个块。
- 文件 '2-雇主责任险.txt'被切分成 1 个块。
- 文件 '3-平安企业团体综合意外险.txt'被切分成 1 个块。
- 文件 '4-雇主安心保.txt'被切分成 1 个块。
- 文件 '5-施工保.txt'被切分成 1 个块。
- 文件 '6-财产一切险.txt'被切分成 1 个块。
- 文件 '7-平安装修保.txt'被切分成 1 个块。
- 文件 '平安产险交通出行意外伤害保险(互联网版)产品说明.pdf'被切分成 1 个块。
- 文件 '平安产险交通工具意外伤害保险(互联网版)条款.pdf'被切分成 3 个块。
- 文件 '平安企业团体综合意外险(互联网版)适用条款.pdf'被切分成 4 个块。
- 跳过空文件: 平安商业综合责任保险(亚马逊).pdf
- 文件 '平安境内紧急医疗救援服务条款.pdf'被切分成 2 个块。

incorrect startxref pointer(1)

- 文件 '平安附加疾病身故保险条款.pdf'被切分成 1 个块。

正在执行向量搜索,查询语句: '工伤保险和雇主险有什么区别? '

--- 向量搜索结果 ---

--- 结果 1 ---

来源文件: 2-雇主责任险.txt

相关度得分 (cosine similarity): 0.8496

内容预览: 【雇主责任险】 Q1 雇员意外事故给企业造成的损失有多大? 甚至可能让一家公司倒闭!! 员工出外勤不幸遇到车祸被撞高位截瘫 公司赔偿近一百万 抽干公司流动资金 无力维持生产 走破产程序... Q2 雇主责任险的保障范围和其他亮点 1. 保障 只要有用工需求的单位,都可以购买雇主责任险。 2. 保障范围: 死亡赔偿金 伤残赔偿金 医疗费用 误工费 (c款不赔) 法律诉讼费 (c款不赔) 3. 其他亮点...

index_and_search_docs-embedding.py

Version4: 添加外部数据源 (AI搜索MCP)

Version4: 添加外部数据源 (AI搜索MCP)

找到之前成功运行 AI 搜索 MCP 的代码，比如 `stock_query_assistant-3.py`，拷贝到当前项目中

```
# MCP 工具配置
tools = [{
    "mcpServers": {
        "tavily-mcp": {
            "command": "npx",
            "args": ["-y", "tavily-mcp@0.1.4"],
            "env": {
                "TAVILY_API_KEY": "tvly-dev-XXXXXXXXXXXX"
            },
            "disabled": False,
            "autoApprove": []
        }
    }
}, 'exc_sql', 'arima_stock']
```

Version4: 添加外部数据源 (AI搜索MCP)

@ai_bot-3.py 添加 tavily-mcp, 参考 @stock_query_assistant-3.py 这里的tavily-mcp工具添加, 这样让bot回答问题的時候, 除了使用rag_cfg, 还可以看到网络上的tavily-mcp数据, 写入到 ai_bot-4.py

```
ai_bot-4.py +96 ✓

# ai_bot-4.py
import os
import asyncio
from typing import Optional
from qwen_agent.agents import Assistant
from qwen_agent.gui import WebUI
import warnings
warnings.filterwarnings("ignore")

def init_agent_service():
    """初始化具备 Elasticsearch RAG 和网络

    # 步骤 1: LLM 配置
    llm_cfg = {
```



```
ai_bot-4.py > init_agent_service

10 def init_agent_service():
41     # MCP 工具配置 - 新增 tavily-mcp
42     tools_cfg = [{
43         "mcpServers": {
44             "tavily-mcp": {
45                 "command": "npx",
46                 "args": ["-y", "tavily-mcp@0.1.4"],
47                 "env": {
48                     "TAVILY_API_KEY": os.getenv('TAVILY_API_KEY', "tvly
49                 },
50                 "disabled": False,
51                 "autoApprove": []
52             }
53         }
54     }]
55
56     # 获取文件夹下所有文件
57     file_dir = os.path.join(os.path.dirname(__file__), 'docs')
58     files = []
59     if os.path.exists(file_dir):
60         for file in os.listdir(file_dir):
61             file_path = os.path.join(file_dir, file)
```

Version4： 添加外部数据源（AI搜索MCP）

施工保主要适用于哪些场景？

施工保主要适用于哪些场景？



user

施工保主要适用于以下场景：

1. **工程自身**：在施工过程中，由于自然灾害或意外事故造成的工程自身财产损失。
2. **三者责任**：因施工过程中发生的意外事故导致周边第三者的人身伤亡或财产损失责任。
3. **员工意外**：施工人员因意外事故导致的人身伤亡所造成的医疗费用支出。

这些保障涵盖了施工过程中可能遇到的主要风险，确保了工程、第三方以及施工人员的安全和权益。



搜索AI新闻

搜索AI新闻



user

► Start calling tool "tavily-mcp-tavily-search" ...

► Finished tool calling.

以下是近期关于AI的一些新闻摘要：

1. **Duolingo转向“AI优先”公司引发争议：**
 - Duolingo计划使用生成式AI自动化一些任务，减少非员工承包商的使用。这引发了公众对公司品牌的负面看法。尽管公司强调AI不会取代其员工，但这种转变仍然引起了担忧。
 - [详细内容](#)
2. **AI检测脑肿瘤准确率达到97%：**
 - Robovision Healthcare和荷兰癌症研究所（NKI）联合开发了一种名为BrainMets AI的解决方案，能够以前所未有的精度检测脑转移性疾病。
 - [详细内容](#)
3. **Meta在作者版权诉讼中胜诉：**
 - 美国地方法院法官Vince Chhabria裁定，作者未能提供足够证据证明Meta的AI会稀释他们的作品市场，因此Meta的行为在美国版权法下是合法的。
 - [详细内容](#)
4. **Meta加强AI团队：**
 - Meta从OpenAI聘请了多位研究人员，包括Trapit Bansal等，以增强其在AI领域的研究力量。
 - [详细内容](#)
5. **两党法案旨在阻止中国AI进入联邦机构：**
 - 一项新的立法提案旨在阻止中国的人工智能系统进入美国联邦机构，以确保美国在全球AI竞

Version5: 界面美化

Gradio使用

Thinking: 什么是Gradio?

Gradio 是一个开源的 Python 库，用于快速搭建交互式 Web 界面。开发者无需掌握复杂的前端技术（如 HTML、CSS、JavaScript），可在几分钟内生成可分享的、美观易用的演示页面。

- **极简开发**：几行代码即可定义组件（文本框、图像上传、音频、视频等），并自动渲染为 Web 界面。
- **多模态支持**：原生支持文本、图像、音频、视频、文件等多种输入/输出类型，适配各类 AI 模型。
- **一键分享**：通过 `share=True` 参数可生成公开链接，方便团队协作、客户演示或教学。
- **高可定制性**：提供 Blocks API 自定义布局，支持 CSS 样式调整，满足个性化需求。
- **与主流框架无缝集成**：兼容 PyTorch、TensorFlow、Hugging Face Transformers 等，可用于 Stable Diffusion WebUI、LLaMa Factory 等热门项目

Gradio使用

Thinking: 什么是Gradio 的 Blocks API ?

Gradio 的 Blocks API 是 Gradio 提供的低级布局 API，它允许开发者像“搭积木”一样自由组合 UI 组件。

上下文管理: 用 `with gr.Blocks() as demo:` 包裹所有组件，形成可自定义的页面结构。

布局组件:

- Row: 水平排列子组件。
- Column: 垂直排列子组件（默认行为）。
- Tab: 创建多标签页界面，每个标签页独立布局。
- Accordion: 可折叠面板，隐藏/展开内容。
- Group: 将相关组件打包为逻辑组，便于样式统一或事件绑定。

灵活性: 可嵌套布局组件（如 Row 内嵌 Column），实现复杂网格或分栏效果

Gradio使用

```
import gradio as gr
with gr.Blocks() as demo:
    with gr.Row(): # 水平行
        with gr.Column(): # 左侧列
            input_text = gr.Textbox(label="输入")
            submit_btn = gr.Button("提交")
        with gr.Column(): # 右侧列
            output_text = gr.Textbox(label="输出")
    submit_btn.click(fn=lambda x: f"你输入了: {x}", inputs=input_text, outputs=output_text)

demo.launch()
```

输入

123

输出

你输入了: 123

提交

block_api_demo1.py

Version5: 界面美化

在 @ai_bot-4.py 的基础上完善gradio界面，参考这个截屏。写入到 ai_bot-5.py

```
ai_bot-5.py +71 -68 ✓

from qwen_agent.agents import Assistant
import warnings
import gradio as gr
import time

warnings.filterwarnings("ignore")

12 hidden lines

# 步骤 2: RAG 配置 - 激活并配置 Elastic
rag_cfg = {
    "rag_backend": "elasticsearch",
    "rag_backend": "elasticsearch",
    "es": {
```



```
ai_bot-5.py > main > on_suggestion_click

107 def main():
108     """启动自定义的 Gradio Web 图形界面"""
109
110     custom_css = """
111     body { font-family: 'Arial', sans-serif; background-color: #F7F8FA;
112     .gradio-container { max-width: 100% !important; background-color: #
113     #sidebar { background-color: #FFFFFF; padding: 20px; border-right:
114     #logo { display: flex; align-items: center; margin-bottom: 30px; }
115     #logo-img { width: 40px; height: 40px; margin-right: 10px; border-r
116     #logo-text { font-size: 24px; font-weight: bold; color: #333; }
117     .sidebar-btn {
118         display: block; width: 100%; text-align: left; padding: 12px 15
119         border: none; background: none; font-size: 16px; margin-bottom:
120         cursor: pointer; border-radius: 8px; color: #374151;
121     }
122     .sidebar-btn.active, .sidebar-btn:hover { background-color: #F3F4F6
123     #main-chat { padding: 20px; background-color: #F7F8FA; }
124     #chat-header { text-align: center; margin-top: 8%; margin-bottom: 4
125     #chat-header-title { font-size: 48px; font-weight: 500; color: #333
126     #chat-header-subtitle { font-size: 16px; color: #6B7280; margin-top
127     #chatbot { box-shadow: none; border: none; background-color: transp
128     .message-bubble-user { background: #DBEAFE !important; color: #1E40
129     .message-bubble-bot { background: #FFFFFF !important; color: #37415
130     .suggestion-row { margin-top: 20px; display: flex; justify-content:
131     .suggestion-btn {
132         background-color: #FFFFFF; border: 1px solid #E5E7EB; padding:
133         border-radius: 18px; cursor: pointer; color: #374151; font-size
134         transition: all 0.2s ease-in-out;
135     }
```

Version5：界面美化



Version5： 界面美化

@ai_bot-5.py 左侧导航应用的LOGO使用项目中的 logo.png

css可以单独用文件保存，结构清晰

目前 知识库管理、收藏、历史功能暂未实现，可以点击图标的时候，提示暂未实现，敬请期待

另外提问后，回答结果是一次性的显示到页面上，需要改成 stream逐字的方式，体验会更好。之前使用qwen-agent默认是可以实现stream显示的。

qwen-agent的源代码也在项目中的 qwen_agent中

以上这些编写新代码，到 ai_bot-6.py

Version5：界面美化



打卡：AI搜索类应用



搭建AI搜索类应用：

- 采用BM25算法，进行关键词匹配，召回候选chunk
- 文档数量不多的时候，可以用qwen-agent自带的retrieval
- 文档数量多的时候，打造自己的 es_retrieval，支持海量文档快速召回
- 添加外部搜索源，比如 tavily-mcp
- 美化界面



CASE：长对话检索与问答

CASE：长对话检索与问答

CASE：长对话检索与问答

利用 Qwen Agent 中的 DialogueRetrievalAgent 实现长对话检索与问答

Step1，定义智能体

通过 `DialogueRetrievalAgent` 实例化对话检索型智能体，配置大模型参数。

Step2，构造长文本场景

- 构造一个包含大量无关内容和一个关键信息的长文本，模拟真实场景下的"噪声干扰"。
- 用户问题和长文本一同作为消息输入。

Step3，智能体推理与检索

智能体自动分析用户问题，从长文本中检索出关键信息并作答。

CASE：长对话检索与问答

DialogueRetrievalAgent 使用

```
from qwen_agent.agents import DialogueRetrievalAgent

# 实例化 Agent
bot = DialogueRetrievalAgent(llm={'model': 'qwen-max'})

# 构造超长对话或文本
long_text = ', '.join(['这是干扰内容'] * 1000 + ['小明的爸爸叫大头'] + ['这是干扰内容'] * 1000)
messages = [{'role': 'user', 'content': f'小明爸爸叫什么？ \n{long_text}'}]

# 推理
for response in bot.run(messages):
    print('bot response:', response)
```

DialogueRetrievalAgent 的适用场景：

- 超长对话、超长文本的问答场景。
- 用户输入内容极长，普通 LLM 无法直接处理时。
- 需要将长对话内容分片存储、检索，再进行问答。

1-long_dialogue.py

CASE: 长对话检索与问答

运行结果:

```
2025-05-27 13:07:34,303 - memory.py - 116 - INFO - {"keywords_zh": ["小明", "爸爸", "叫"], "keywords_en": ["Xiaoming", "father", "name"], "text": "小明爸爸叫什么? "}
2025-05-27 13:07:35,556 - simple_doc_parser.py - 413 - INFO - Start parsing workspace\dialogue_history__20250527_130731.txt...
2025-05-27 13:07:35,562 - simple_doc_parser.py - 461 - INFO - Finished parsing workspace\dialogue_history__20250527_130731.txt. Time spent: 0.005738258361816406 seconds.
2025-05-27 13:07:35,564 - doc_parser.py - 108 - INFO - Start chunking workspace\dialogue_history__20250527_130731.txt (dialogue_history__20250527_130731.txt)...
2025-05-27 13:07:35,565 - doc_parser.py - 126 - INFO - Finished chunking workspace\dialogue_history__20250527_130731.txt (dialogue_history__20250527_130731.txt). Time spent: 0.0 seconds.
2025-05-27 13:07:35,567 - base_search.py - 56 - INFO - all tokens: 8007
2025-05-27 13:07:35,568 - base_search.py - 59 - INFO - use full ref
bot response: [{'role': 'assistant', 'content': '小', 'reasoning_content': '', 'extra': {'model_service_info': {'status_code': <HTTPStatus.OK: 200>, 'request_id': 'b6f524bb-ca42-930c-a892-e26a02c8621e', 'code': '', 'message': '', 'output': {'text': None, 'choices': [{'finish_reason': 'null', 'message': {}}], 'finish_reason': None}, 'usage': {'input_tokens': 8095, 'output_tokens': 1}}}}]
bot response: [{'role': 'assistant', 'content': '小明的', 'reasoning_content': '', 'extra': {'model_service_info': {'status_code': <HTTPStatus.OK: 200>, 'request_id': 'b6f524bb-ca42-930c-a892-e26a02c8621e', 'code': '', 'message': '', 'output': {'text': None, 'choices': [{'finish_reason': 'null', 'message': {}}], 'finish_reason': None}, 'usage': {'input_tokens': 8095, 'output_tokens': 3}}}}]
bot response: [{'role': 'assistant', 'content': '小明的爸爸叫大头。', 'reasoning_content': '', 'extra': {'model_service_info': {'status_code': <HTTPStatus.OK: 200>, 'request_id': 'b6f524bb-ca42-930c-a892-e26a02c8621e', 'code': '', 'message': '', 'output': {'text': None, 'choices': [{'finish_reason': 'null', 'message': {}}], 'finish_reason': None}, 'usage': {'input_tokens': 8095, 'output_tokens': 8}}}}]
bot response: [{'role': 'assistant', 'content': '小明的爸爸叫大头。', 'reasoning_content': '', 'extra': {'model_service_info': {'status_code': <HTTPStatus.OK: 200>, 'request_id': 'b6f524bb-ca42-930c-a892-e26a02c8621e', 'code': '', 'message': '', 'output': {'text': None, 'choices': [{'finish_reason': 'stop', 'message': {}}], 'finish_reason': None}, 'usage': {'input_tokens': 8095, 'output_tokens': 8}}}}]
bot response: [{'role': 'assistant', 'content': '小明的爸爸叫大头。', 'reasoning_content': '', 'extra': {'model_service_info': {'status_code': <HTTPStatus.OK: 200>, 'request_id': 'b6f524bb-ca42-930c-a892-e26a02c8621e', 'code': '', 'message': '', 'output': {'text': None, 'choices': [{'finish_reason': 'stop', 'message': {}}], 'finish_reason': None}, 'usage': {'input_tokens': 8095, 'output_tokens': 8}}}}]
```

CASE: 多文档并行问答

CASE：多文档并行问答

CASE：多文档并行问答

利用 Qwen Agent 框架中的 ParallelDocQA 实现多文档/大文档的并行问答与内容检索

Step1，定义智能体

通过 ParallelDocQA 实例化文档问答型智能体，配置大模型参数。

Step2，构造多模态输入场景

- 用户输入既可以包含文本问题，也可以包含文件（如 PDF、Word、PPT、TXT、HTML 等）。
- 支持多种文件类型，适合复杂的知识检索和问答场景。

Step3，智能体并行检索与问答

- 智能体自动对输入的每个文件并行进行内容检索和问答。
- 支持 RAG增强（二次精确关键词召回），提升答案准确率。

CASE：多文档并行问答

ParallelDocQA 使用

```
from qwen_agent.agents.doc_qa import ParallelDocQA

# 实例化 Agent
bot = ParallelDocQA(llm={'model': 'qwen2.5-72b-instruct', 'generate_cfg':
{'max_retries': 10}})

# 构造带文件的消息
messages = [
    {
        'role': 'user',
        'content': [
            {'text': '介绍实验方法'},
            {'file': 'https://arxiv.org/pdf/2310.08560.pdf'},
        ]
    },
]
```

```
# 推理
for rsp in bot.run(messages):
    print('bot response:', rsp)
```

ParallelDocQA 的适用场景：

- 支持多文档、多格式（PDF/Word/PPT/TXT/HTML）并行检索与问答。
- 适合大体量文档、复杂结构材料的高效问答。
- 需要并行处理、RAG召回、自动摘要等场景。

2-parallel_doc_qa.py

CASE：多文档并行问答

模型的效果如何？

 QWEN TECHNICAL REPORT.pdf

```
2025-05-27 13:16:46,829 - simple_doc_parser.py - 461 - INFO - Finished parsing C:\Users\cheny\AppData\Local\Temp\gradio\2d97664a0e128993adde1fd8467b05c80c50ab7c94810929821846c634c5871b\QWEN_TECHNICAL_REPORT.pdf. Time spent: 5.847546815872192 seconds.
2025-05-27 13:16:46,837 - doc_parser.py - 108 - INFO - Start chunking C:\Users\cheny\AppData\Local\Temp\gradio\2d97664a0e128993adde1fd8467b05c80c50ab7c94810929821846c634c5871b\QWEN_TECHNICAL_REPORT.pdf (QWEN_TECHNICAL_REPORT.pdf)...
2025-05-27 13:16:46,844 - doc_parser.py - 126 - INFO - Finished chunking C:\Users\cheny\AppData\Local\Temp\gradio\2d97664a0e128993adde1fd8467b05c80c50ab7c94810929821846c634c5871b\QWEN_TECHNICAL_REPORT.pdf (QWEN_TECHNICAL_REPORT.pdf). Time spent: 0.006867170333862305 seconds.
2025-05-27 13:16:46,847 - parallel_doc_qa.py - 200 - INFO - Parallel Member Num: 68
2025-05-27 13:20:49,464 - parallel_doc_qa.py - 210 - INFO - Finished parallel_exec. Time spent: 242.6156198978424 seconds.
2025-05-27 13:20:51,615 - parallel_doc_qa.py - 113 - INFO - {"keywords_zh": ["实验方法", "方法"], "keywords_en": ["experimental method", "method"]}
2025-05-27 13:20:52,639 - simple_doc_parser.py - 411 - INFO - Read parsed C:\Users\cheny\AppData\Local\Temp\gradio\2d97664a0e128993adde1fd8467b05c80c50ab7c94810929821846c634c5871b\QWEN_TECHNICAL_REPORT.pdf from cache.
2025-05-27 13:20:52,640 - doc_parser.py - 108 - INFO - Start chunking C:\Users\cheny\AppData\Local\Temp\gradio\2d97664a0e128993adde1fd8467b05c80c50ab7c94810929821846c634c5871b\QWEN_TECHNICAL_REPORT.pdf (QWEN_TECHNICAL_REPORT.pdf)...
2025-05-27 13:20:52,680 - doc_parser.py - 126 - INFO - Finished chunking C:\Users\cheny\AppData\Local\Temp\gradio\2d97664a0e128993adde1fd8467b05c80c50ab7c94810929821846c634c5871b\QWEN_TECHNICAL_REPORT.pdf (QWEN_TECHNICAL_REPORT.pdf). Time spent: 0.03987622261047363 seconds.
2025-05-27 13:20:52,685 - base_search.py - 56 - INFO - all tokens: 70893
Building prefix dict from the default dictionary ...
Loading model from cache C:\Users\cheny\AppData\Local\Temp\jieba.cache
Loading model cost 0.482 seconds.
Prefix dict has been built successfully.
```



实验方法主要包括以下几个方面：

- 1. 数据：
 - 数据集的设计确保了数据的多样性和广泛性，涵盖了多种类型、领域和任务。数据集包括公共网络文档、百科全书、书籍、代码等，并且是多语言的，主要包含英语和中文。
 - 数据集的大小对开发强大的大型语言模型至关重要。最终词汇表的大小约为152K。
- 2. 模型设计与扩展：
 - 模型架构方面，选择了SwiGLU作为激活函数，这是一种结合了Swish和Gated Linear Unit (GLU)的激活函数。实验表明，基于GLU的激活函数通常优于其他基线选项，如GeLU。
 - 模型的上下文长度进行了扩展，以支持更长的文本理解和生成。训练时使用了2048的上下文长度。
- 3. 训练：
 - 训练采用了标准的自回归语言建模方法，即根据前文预测下一个词。
 - 使用了AdamW优化器进行预训练优化，设置超参数为 $\beta_1 = 0.9$, $\beta_2 = 0.95$, $\epsilon = 10^{-8}$ 。
 - 学习率采用余弦学习率调度，并设置了每个模型大小的峰值学习率。
 - 所有模型均使用BFloat16混合精度进行训练，以提高训练稳定性。
- 4. 评估：
 - 通过一系列基准测试评估模型的零样本和少样本学习能力，包括MMLU、C-Eval、GSM8K、MATH、HumanEval、MBPP和BBH等。
 - 与最新的开源基础模型进行比较，如LLaMA、LLaMA2、MPT、Falcon、Baichuan2、ChatGLM2、InternLM、XVERSE和StableBeluga2。

这些方法确保了QWEN模型在多个基准测试中的优越性能。



CASE：多文档并行问答

介绍实验方法

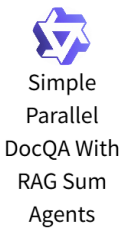
 QWEN TECHNICAL REPORT.pdf

```
2025-05-27 13:26:10,435 - parallel_doc_qa.py - 180 - INFO - user_question: 模型的效果如何

2025-05-27 13:26:10,439 - doc_parser.py - 93 - INFO - Read chunked C:\Users\cheny\AppData\
ocal\Temp\gradio\2d97664a0e128993adde1fd8467b05c80c50ab7c94810929821846c634c5871b\QWEN_TI
NICAL_REPORT.pdf from cache.
2025-05-27 13:26:10,441 - parallel_doc_qa.py - 200 - INFO - Parallel Member Num: 68
2025-05-27 13:26:28,925 - parallel_doc_qa.py - 210 - INFO - Finished parallel_exec. Time
ent: 18.48313570022583 seconds.
2025-05-27 13:26:33,928 - parallel_doc_qa.py - 113 - INFO -
{
  "keywords_zh": ["模型", "效果", "性能", "基准测试", "QWEN", "GPT-4", "LLaMA2", "CODE-Q",
  "MATH-QWEN-CHAT", "RLHF", "SFT", "准确率", "数据集", "评估", "对比"],
  "keywords_en": ["model", "effectiveness", "performance", "benchmark", "QWEN", "GPT-4",
  LaMA2", "CODE-QWEN", "MATH-QWEN-CHAT", "RLHF", "SFT", "accuracy", "dataset", "evaluation",
  "comparison"]
}

2025-05-27 13:26:33,934 - doc_parser.py - 93 - INFO - Read chunked C:\Users\cheny\AppData\
ocal\Temp\gradio\2d97664a0e128993adde1fd8467b05c80c50ab7c94810929821846c634c5871b\QWEN_TI
NICAL_REPORT.pdf from cache.
2025-05-27 13:26:33,935 - base_search.py - 56 - INFO - all tokens: 70893
```

68 个 chunk 说明分块粒度较细。18.5 秒是所有 chunk 并行问答的总耗时（包括模型推理、IO 等）。



QWEN模型的效果非常显著，具体表现在以下几个方面：

1. 基础语言模型：

- QWEN基础语言模型经过大量数据训练，包括多达3万亿个token的文本和代码，涵盖了广泛的领域。这些模型在多个下游任务中表现出色，甚至在某些任务上超越了更大规模的模型。

2. 对话模型：

- QWEN-CHAT模型经过精心微调，使用了监督微调（SFT）和强化学习与人类反馈（RLHF）等技术。基准测试显示，SFT模型表现优异，而RLHF模型能够生成更符合人类偏好的响应。通过人类评价，RLHF模型在复杂任务上的表现非常有竞争力，尽管仍略逊于GPT-4。

3. 编码模型：

- 专门的编码模型CODE-QWEN及其对话版本CODE-QWEN-CHAT在代码理解、生成、调试和解释方面表现出色。在HumanEval、MBPP和HumanEvalPack等基准测试中，这些模型展示了高水平的专业能力。

4. 数学模型：

- 数学模型MATH-QWEN-CHAT在处理数学问题时表现出色，尽管与专有模型相比仍有差距，但明显优于开源模型。

5. 工具使用和代理：

- QWEN模型在工具选择和输入方面表现出色，尤其是在识别查询与可用工具的相关性方面。随着模型规模的增加，准确率逐渐提高，但在某些情况下，性能提升有限。

总体而言，QWEN系列模型在多个领域和任务中表现出色，具有很强的竞争力。



CASE：多文档并行问答

ParallelDocQA 核心流程：

Step1. 文件解析与分块

- `_get_files(messages)`：提取用户消息中的所有有效文件（支持 PDF/Word/PPT/TXT/HTML 等）。
- `_parse_and_chunk_files(messages)`：对每个有效文件进行解析，并按设定 chunk size（如 1000 tokens）分块，形成多个文档片段。

Step2. 并行分片问答

主流程 `_run(messages, lang, **kwargs)`：

- 提取用户问题（`user_question`）。
- 对所有文档分块，生成每个 chunk 的 QA 任务（`data`）。
- 通过 `parallel_exec` 并行调用 `_ask_member_agent`，对每个 chunk 进行独立问答。
- 多轮重试机制，过滤无效/空响应。
- 汇总所有 chunk 的有效回答（`member_res`）。

CASE：多文档并行问答

Step3. 检索与摘要

`_retrieve_according_to_member_responses(...)` :

- 对所有 `chunk` 的回答进行关键词生成（`GenKeyword`），构造检索 `query`。
- 调用 `retrieval` 工具，对原始文档再次检索，召回最相关内容。
- 格式化为知识片段，供后续摘要。

`self.summary_agent.run(...)` : 汇总检索到的内容，调用摘要 `Agent` 生成最终答案。

`ParallelDocQA`的并行处理机制：

- 采用 `parallel_exec` 对每个文档 `chunk` 并行分发 `QA` 任务，大幅提升处理效率。
- 每个 `chunk` 独立调用 `ParallelDocQAMember` 进行问答，互不影响。
- 支持多轮重试，自动过滤无效响应，保证结果质量。

CASE：多文档并行问答

Thinking: Parallel Member Num=68 是每一个chunk，都要经过一次LLM进行回答么？

是的，Parallel Member Num: 68 表示：文档被分成了 68 个分片（chunk）。

每个分片 都会独立调用一次 LLM（即 68 次 LLM 问答），每次问答的内容是“用户问题 + 当前分片的内容”。

这些 68 次问答是并行进行的（通过 parallel_exec），大大提升了处理速度。

CASE：多文档并行问答

ParallelDocQA 具体执行流程：

Step1，分片并行问答

每个 chunk 由 `_ask_member_agent` 调用 LLM 进行独立问答，得到 68 个回答结果。

Step2，结果收集与过滤

收集所有分片的回答，过滤掉无效或空响应，只保留有用的答案片段。

Step3，结果汇总与检索

将所有有效分片的回答拼接、汇总，作为 “`member_res`”。

后续还会用这些回答生成检索关键词，再对原始文档做一次更精准的检索（RAG召回）。

Step4，最终摘要

最后将检索到的内容交给摘要 Agent，生成最终的综合答案。

每个分片一次 LLM 问答，全部并行，总数等于分片数。

所有分片的结果会被汇总、过滤、再进一步检索和摘要，最终输出给用户。

CASE：多文档并行问答

Thinking: ParallelDocQA 相比于传统DocQA的优势是什么？

1. 并行分片问答提升召回覆盖率

传统的文档问答/RAG流程，通常是先检索少量相关片段，再让大模型生成答案。如果检索阶段召回不充分，后续 LLM 回答就会受限于有限的上下文，导致结果不全或不精准。

parallel_doc_qa 会将文档分成大量小片段，对每个片段都独立发起一次 LLM 问答（并行处理），极大提升了召回的覆盖面，不会遗漏文档中的重要信息。

2. 多轮过滤与再检索提升精准度

并行问答后，系统会过滤无效/空响应，只保留有用的答案片段。

然后会用这些片段生成更精准的检索关键词，对原始文档再次检索（RAG召回），进一步提升召回的准确性。

最后还会用摘要 Agent 对召回内容进行综合归纳，输出更高质量的答案。

3. 适合大文档/多文档/复杂结构

这种机制特别适合大体量、多格式、结构复杂的文档，能保证信息不遗漏，且结果更全面、精准。

CASE: 多智能体问答

CASE：多智能体问答

CASE：多智能体问答

利用 Qwen Agent 实现多智能体协作与 @mention（@提及）功能

Step1，定义多智能体

包含代码解释器（ReActChat）、文档问答（BasicDocQA）、通用助手（Assistant）三类智能体。

Step2，初始化服务

通过 `init_agent_service` 返回智能体列表。

Step3，多智能体协作

- 用户输入被解析后，自动路由到对应的智能体进行处理。
- 支持多轮对话和多智能体并存。

ReActChat 使用

ReActChat 使用

```
from qwen_agent.agents import ReActChat
react_chat_agent = ReActChat(
    llm={'model': 'qwen-max'},
    name='代码解释器',
    description='代码解释器，可用于执行Python代码。',
    system_message='you are a programming expert...'
    function_list=['code_interpreter'],
)
```

ReActChat 的适用场景：

- 适用于需要 **工具调用+推理** 场景，如代码解释、数据分析等。
- 支持 `function_list` 配置（如 `code_interpreter`），可自动调用外部工具。
- 支持 `system_message` 指定智能体角色和风格。

BasicDocQA 使用

BasicDocQA 使用

```
from qwen_agent.agents.doc_qa import BasicDocQA
doc_qa_agent = BasicDocQA(
    llm={'model': 'qwen-max'},
    name='文档问答',
    description='根据用户输入的问题和文档，从文档中找到答案',
)
```

BasicDocQA的适用场景：

适用于 **文档问答** 场景，支持根据用户问题和上传文档自动检索答案。

CASE：多智能体问答

多智能体初始化与注册

```
def init_agent_service():  
    llm_cfg = {'model': 'qwen-max'}  
  
    react_chat_agent = ReActChat(  
        llm=llm_cfg,  
        name='代码解释器',  
        description='代码解释器，可用于执行Python代码。',  
        system_message='you are a programming expert, skilled in  
writing code to solve mathematical problems and data analysis  
problems.',  
        function_list=['code_interpreter'],  
    )
```

```
doc_qa_agent = BasicDocQA(  
    llm=llm_cfg,  
    name='文档问答',  
    description='根据用户输入的问题和文档，从文档中找到答案',  
)  
assistant_agent = Assistant(llm=llm_cfg, name='小助理',  
description="I'm a helpful assistant")  
return [react_chat_agent, doc_qa_agent, assistant_agent]
```

统一初始化多种类型智能体，便于后续协作和路由。

CASE: 多智能体问答

@代码解释器 $2^{10} = ?$

Thought: 我们需要计算2的10次方。这可以通过使用Python的指数运算符 (`**`) 来完成。

Action: code_interpreter

Action Input:

py

Calculate 2 to the power of 10

```
result = 2 ** 10
```

```
result
```

Observation: execute_result:

1024

Thought: 我得到了最终的答案

Final Answer: 2的10次方等于1024。



代码解释器



CASE：多智能体问答

@文档问答 这篇论文解决了什么问题？

 QWEN TECHNICAL REPORT.pdf

这篇论文介绍了QWEN，这是阿里巴巴集团开发的一系列大型语言模型中的第一个。QWEN是一个全面的语言模型系列，包括具有不同参数数量的不同模型。它包括基础的预训练语言模型QWEN，以及通过人类对齐技术进行微调的聊天模型QWEN-CHAT。这些模型在众多下游任务中表现出色，并且特别地，使用从人类反馈中学习的强化学习（RLHF）训练的聊天模型在复杂任务（如使用代码解释器）上表现出令人印象深刻的表现，甚至可以和更大的模型相媲美。

此外，还开发了专门针对编码和数学问题的模型CODE-QWEN、CODE-QWEN-CHAT 和 MATH-QWEN-CHAT，它们在比较基准数据集上的表现优于开源模型，并且略逊于专有模型。这篇论文也解决了大型语言模型的可重复性、可控性和服务提供商的可访问性等问题。



文档问答



CASE：多智能体问答

@小助理 你好！



小助理

你好！有什么我可以帮助你的吗？如果你有任何问题或需要了解更多信息，请告诉我。



可以通过 @mention 切换不同的智能体进行回答

Assistant 使用

Assistant 使用

```
from qwen_agent.agents.assistant import Assistant

assistant_agent = Assistant(llm=llm_cfg, name='小助理', description="I'm a helpful assistant")
```

Assistant 定义

```
Assistant(
    function_list: Optional[List[Union[str, Dict, BaseTool]]] = None, # 工具/函数列表
    llm: Optional[Union[Dict, BaseChatModel]] = None,                # LLM配置或实例
    system_message: Optional[str] = DEFAULT_SYSTEM_MESSAGE,         # 系统提示词
    name: Optional[str] = None,                                       # 智能体名称
    description: Optional[str] = None,                                # 智能体描述
    files: Optional[List[str]] = None,                                # 关联知识文件
    rag_cfg: Optional[Dict] = None                                    # RAG相关配置
)
```

Assistant是 Qwen-Agent 中通用性极强的智能体，集成了 RAG能力和工具调用（工具调用）能力，适用于大多数问答、知识检索、工具调用等场景。

- 支持外部知识检索（RAG），可结合上传文档、知识库等进行问答。
- 支持函数/工具调用，具备插件式扩展能力。
- 可自定义系统提示词、名称、描述、知识文件等。
- 继承自 FnCallAgent，具备其全部能力。

rag_cfg 参数详解

rag_cfg 参数详解

用于控制 RAG相关的高级行为，影响知识检索的分片、召回、关键词生成等细节。

- `max_ref_token`: int, 检索时单次最大参考 token 数，决定召回内容的窗口大小。默认 20000。
- `parser_page_size`: int, 文档分片时每片最大 token 数，影响分片粒度。默认 500。
- `rag_keygen_strategy`: str, 检索关键词生成策略，可选：
 - `GenKeyword`: 直接生成关键词（默认）
 - `SplitQueryThenGenKeyword`: 先拆分问题再生成关键词
 - `GenKeywordWithKnowledge`: 结合"问题内容+参考资料词表"进行关键词生成
- `rag_searchers`: list, 检索子策略列表，如 `['keyword_search', 'front_page_search']`，支持多种召回融合。

rag_cfg 参数详解

rag_cfg 配置示例

```
rag_cfg = {  
    'max_ref_token': 4000, # 单次最大召回 token 数  
    'parser_page_size': 500, # 分片粒度  
    'rag_keygen_strategy': 'SplitQueryThenGenKeyword', # 关键词生成策略  
    'rag_searchers': ['keyword_search', 'front_page_search'] # 检索方式  
}
```

通过 `rag_cfg` 可以灵活调整 RAG 检索的窗口、分片、关键词生成和召回方式，适配不同规模和类型的知识库。比如大文档可调大 `parser_page_size`，召回更长上下文；多策略融合可提升召回覆盖率。若不指定 `rag_cfg`，则采用默认配置，适合大多数通用场景。

CASE: 多智能体协作路由

CASE：多智能体协作路由

CASE：多智能体协作路由

利用 Qwen Agent 框架中的 Router 实现多智能体协作与自动路由，支持文本、图片、文件等多模态输入，自动分发给最合适的子智能体处理。

Step1，定义多智能体

包含多模态助手（Assistant，支持图片理解）、工具助手（ReActChat，支持画图和代码解释）等。

Step2，初始化 Router 路由器

通过 Router 聚合多个子智能体，自动根据输入内容分发任务。

Step3，自动路由与多智能体协作

- 用户输入后，Router 自动判断最合适的子智能体进行处理。
- 支持多轮对话和多模态内容的智能分发。

Router 使用

Router 使用

```
def init_agent_service():  
    # 配置大模型  
    llm_cfg = {'model': 'qwen-max'}  
    llm_cfg_vl = {'model': 'qwen-vl-max'}  
    tools = ['image_gen', 'code_interpreter']  
  
    # 定义多模态助手  
    bot_vl = Assistant(llm=llm_cfg_vl, name='多模态助手',  
description='可以理解图像内容。')  
  
    # 定义工具助手  
    bot_tool = ReActChat(  
        llm=llm_cfg,
```

```
        name='工具助手',  
        description='可以使用画图工具和运行代码来解决问题',  
        function_list=tools,  
    )  
  
    # 定义路由器  
    bot = Router(  
        llm=llm_cfg,  
        agents=[bot_vl, bot_tool],  
    )  
    return bot
```

CASE：多智能体协作路由

帮我解释下这张图片



这张图片展示了一位女性和一只狗在海滩上的温馨互动场景。

1.背景环境

- 地点：图片拍摄于一个海滩上，沙滩细腻，海浪轻轻拍打着岸边。
- 时间：从光线来看，这可能是日落时分，阳光柔和，给整个画面增添了一种温暖的氛围。

2.人物与动物

- 女性：她坐在沙滩上，穿着格子衬衫和深色裤子，显得休闲而放松。她的表情非常开心，正在与狗进行互动。
- 狗：这是一只金毛犬，戴着彩色的项圈，看起来非常友好和温顺。它正用前爪轻轻地触碰女性的手，仿佛在“握手”。

3.互动细节

- 动作：狗用前爪与女性的手相触，这种动作通常表示友好和信任。女性则微笑着回应，显示出她对狗的喜爱和享受这一刻的心情。
- 情感：整个场景充满了温馨和欢乐，展现了人与宠物之间的深厚感情。

4.整体氛围

- 温暖：夕阳的光线洒在沙滩上，给人一种温暖和宁静的感觉。
- 和谐：人与自然、人与宠物之间的和谐共处，体现了生活的美好和简单快乐。

这张图片不仅展示了美丽的自然风光，更传递了人与动物之间真挚的情感交流，令人感到温馨和愉悦。



多模态助手



CASE：多智能体协作路由

帮我画一个小女孩和小狗在沙滩上玩的照片



Call: 工具助手



Thought: 我需要使用image_gen工具来生成一张根据描述的小女孩和小狗在沙滩上玩耍的图片。

Action: image_gen

Action Input: {"prompt": "a little girl and a puppy playing on the beach, with the sunset in the background, creating a warm and joyful atmosphere"}

Observation: {"image_url":

"https://image.pollinations.ai/prompt/a%20little%20girl%20and%20a%20puppy%20playing%20on%20the%20beach%2C%20with%20the%20sunset%20in%20the%20background%2C%20creating%20a%20warm%20and%20joyful%20atmosphere"}

Thought: 我已获得了一张根据描述生成的小女孩和小狗在沙滩上玩耍的图片。

Final Answer: 这是根据您的描述生成的小女孩和小狗在沙滩上玩耍的照片，背景是日落，营造出一种温暖而欢乐的氛围。





Thank You
Using data to solve problems